VECTOR



Integration of E2E Protection in MICROSAR Working with E2ePw and E2eXf

User Manual

Jan Toennemann

V1.2.1 | Released

Document Information

History

Author	Date	Version	Remarks
Maximilian Hempe	2017-07-11	0.1.0	Initial Document
	2017-10-26	0.2.0	Add description about Developer and Cfg 5 workflow
Yves Plaindoux	2017-10-26	0.3.0	Add E2E Transformer workflow
Yves Plaindoux	2017-11-03	0.4.0	Add Transformer Return Value
Maximilian Hempe	2018-02-18	0.4.1	Changed format for Vector Knowledge base document
Maximilian Hempe	2018-03-27	1.0.0	Released
Yves Plaindoux	2018-04-18	1.0.1	Meaning of Disabled added
Maximilian Hempe	2020-02-10	1.0.2	variant handling for E2ePw, generation and debugging steps for E2ePw
Maximilian Hempe	2020-03-24	1.0.3	Reworked
Anurag Bajpai	2020-08-25	1.1.0	Added chapter for importing E2e Transformer config via dbc
Divyakanth Mullangi	2020-09-11	1.1.1	Added Chapter on How to bring in legacy LDF for E2ePw configuration.
Anurag Bajpai	2021-03-11	1.1.2	Update of screenshots showing E2eXf config via dbc
Maximilian Hempe	2021-03-11	1.1.3	Update of variant handling via JavaScript using E2ePw
Klaus Emmert	2021-04-22	1.1.4	Bugfix in TOC
Klaus Emmert	2021-05-17	1.1.5	Bugfix missing illus
Klaus Emmert	2021-05-26	1.1.6	Bugfix
Klaus Emmert	2021-05-27	1.1.7	Bugfix
Klaus Emmert, Chris Pingel	2021-06-24	1.1.8	Screenshots Reworked
Abdul Rahman Fadloun	2021-07-07	1.1.9	Added chapter Using E2ePw with OEM ARXML file
Klaus Emmert, Jan Toennemann	2022-04-19	1.2.0	Convert to User Manual
Klaus Emmert	2022-08-09	1.2.1	SIP→ BSW Package

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR System Template (ASR 4.2.2)	accessed on 2017-11-07)
[2]	AUTOSAR	E2E_AUTOSAR_SWS_E2ELibrary	AR 4.2.2
[3]	AUTOSAR	AUTOSAR_SWS_E2ETransformer	AR 4.2.2
[4]	Vector	TechnicalReference_E2E.pdf	
[5]	Vector	TechnicalReference_E2EXf.pdf	
[6]	Vector	UserManual_E2EPW.pdf	
[7]	Vector	TechnicalReference_Rte.pdf	
[8]	Vector	TechnicalReference_GenTool_CsAsrLegacyDb2SystemDescr_Vector.pdf	

Caution
This document gives an overview on product-wide behavior or use-cases. To ensure correct operation of the product always follow the instructions provided in the technical references and (additionally for safety projects) the safety manual. The content of these documents always replaces/overrides the content from this user manual.

Contents

1 Introduction and Explanation About this Document 8

2 E2E Protection Wrapper Integration 9

2.1 Configuration of E2E Protection in DaVinci Developer Classic 9

2.1.1 Create DaVinci Developer Classic Workspace Based on your ECUExtract 9

2.1.2 Add a SWC and Port Prototype to the DaVinci Developer Classic Workspace 11

2.1.3 Configure E2E Configuration within the DaVinci Developer Classic 15

2.1.4 Importing E2e Configuration to the MICROSAR Project (DaVinci Developer Classic Workspace is Connected to DaVinci Configurator Classic Project) 19

2.1.5 Optional: Exporting E2e Configuration from DaVinci Developer Classic (e.g. to import it to DaVinci Configurator Classic Project) 19

2.2 Generating the E2E Configuration and the Code in DaVinci Configurator Classic 22

2.3 Debugging in Case the Code Generation Step Was Not Executed Successfully 24

2.4 Using E2ePw With OEM ARXML File 24

2.4.1 Using E2ePw With Variants 25

2.4.1.1 Use the JavaScript AdaptSystemSignalCommunicationForE2EPW 25

2.4.1.2 Add the JavaScript to Your Project 25

2.4.1.3 Execute the JavaScript 25

2.4.1.4 Results of Executing the JavaScript 27

2.4.1.5 Rework DaVinci Developer Classic Workspace 27

2.4.2 Using E2ePw With FanIn/FanOuts ISignal Input File (Single Project) 30

2.4.2.1 What Means FanIn/FanOuts ISignals 30

2.4.2.2 Use the JavaScript AdaptSystemSignalCommunicationForE2EPW 31

2.4.2.3 Add the JavaScript to Your Project 31

2.4.2.4 Execute the JavaScript 32

2.4.2.5 Results of Excuting the JavaScript 32

2.4.2.6 Rework DaVinci Developer Classic Workspace 33

2.5 Using E2ePw With Variants in Signal Mapping 34

2.5.1 Step 1: Enable End-to-End Protection 34

2.5.1.1 Creating a Data Mapping with an E2E Protection 35

2.5.2 Step 2: Create Invariant System Extracts for All Four Variants 37

2.5.2.1 Filtering of the Exported System Extracts 37

2.5.3 Step 3: Create Configuration Files for the E2EPW Generator 40

2.5.4 Step 4: Generate the E2E Protection Wrappers 40
2.6 Configuration of E2EPW Using an LDF File 41
2.6.1 Step 1: Identifying Signals from a LIN Frame to be E2E Protected 41
2.6.2 (Optional) Step 2: Converting LDF Into AUTOSAR SystemDescription ARXML 43
2.6.3 Step 3: Configuration of E2E Using DaVinci Developer Classic 45
2.7 AUTOSAR Specification of E2E Protection (AUTOSAR Classic 4.2.2) 46
2.8 Practical Example Using a Manually Created Configuration 47

3 E2E Transformer Integration 50
3.1 Configuration using Different Input files 50
3.1.1 E2EXf configuration using ECU extract (preferred) 50
3.1.2 Configuration of E2eXf Using a DBC File 51
3.1.3 Resulting System Description and Required configuration 55
3.2 Code Generation 58
3.2.1 Tx Side 59
3.2.1.1 For the ComXf Function 60
3.2.1.2 For the E2EXf Function 60
3.2.2 Rx Side 61
3.2.2.1 For the E2EXf Function 62
3.2.2.2 For the ComXf Function 62
3.3 Transformer Error Handling – Return Value 63
3.4 Abbreviations 65

4 Glossary and Abbreviations 66
4.1 Glossary 66
4.2 Abbreviations 66

5 Contact 67

Illustrations

Figure 2-1 Create New Workspace in DaVinci Developer Classic and Select AR Version of Workspace Files 9
Figure 2-2 Modifying ECU Extract to Fulfill AR Scheme Requirements 10
Figure 2-3 Import of ECU Extract in DaVinci Developer Classic Workspace 10
Figure 2-4 Import Dialog for Importing ECU Extract into DaVinci Developer Classic Workspace 11
Figure 2-5 Adding Necessary Application Component Type in DaVinci Developer Classic 11
Figure 2-6 Configuring the Application Component 12
Figure 2-7 Selecting Software Design in DaVinci Developer Classic 12
Figure 2-8 Creating the SWC Prototype in DaVinci Developer Classic 12
Figure 2-9 Runnable and Trigger for The New SWC 13
Figure 2-10 Switching to Data Mapping in DaVinci Developer Classic Workspace 13
Figure 2-11 Network Signals, Mapping Network Signals to SWC Type,Creating Delegation Ports to Map Network Signals 14
Figure 2-12 Add Port Access to Previously Created Runnable 15
Figure 2-13 Created Delegation Port for Mapping of Network Signal 15
Figure 2-14 Activating End-To-End Protection for the Port Prototype 16
Figure 2-15 Adapt Connected Port Prototypes and Set Port Prototype Adaptations 16
Figure 2-16 Select Element for E2E Configuration and Add E2E Configuration to Delegation Port 16
Figure 2-17 Example Configuration to Enable E2E Protection 17
Figure 2-18 E2E Parameters to NOT Have E2ePw Enabled 18
Figure 2-19 Illustration Showing How DaVinci Developer Classic and DaVinci Configurator Classic are Communicating With Each Other 19
Figure 2-20 Exporting Configuration for Usage in DaVinci Configurator Classic Project 20
Figure 2-21 XML Export Dialog With Selection of AR Version 20
Figure 2-22 Open Input Files of Your DaVinci Configurator Classic Project 21
Figure 2-23 Add the DaVinci Developer Classic Export to Your DaVinci Configurator Classic Input System Description Files 21
Figure 2-24 Select the Previously Exported File 21
Figure 2-25 Update Your Configuration in DaVinci Configurator Classic 21
Figure 2-26 Generating the E2ePw Configuration and Code 22
Figure 2-27 Enable Generation Results View in DaVinci Configurator Classic 23
Figure 2-28 Results From Generation: Configuration File (E2ePw.cfg) and E2ePw Code Files 23
Figure 2-29 Console Output of the E2ePw Generation Step 24
Figure 2-30 Executing the JavaScript " AdaptSystemSignalCommunicationForE2EPW " 26
Figure 2-31 Arguments to Execute the JavaScript AdaptSystemSignalCommunicationForE2EPW 26
Figure 2-323 Output File and Original File After Using the Script 27
Figure 2-33 DaVinci Developer Classic Create Port From Signal View 29
Figure 2-34 Invoking Vector Legacy Converter from Command Line Window with VSDE File as one of the Parameters 44
Figure 2-35 Comparison of ECU Extracts with VSDE Extension File as one of Parameters to Legacy Converter 44
Figure 2-36 EndtoEndProtection for COM IPdus 46
Figure 2-37 AR-Package EndToEndProtections 48
Figure 2-38 Configuration for an EndToEndProtectionDescription 48
Figure 2-39 Configuration for EndToEndProtectionSignalIPdu 49

Figure 3-1 AR Configuration for an E2E Transformer ISignal or ISignalGroup 50
Figure 3-2 DB Configuration for the Attributes for E2E Profile 05 51
Figure 3-3 Verifying\Modifying the Attributes for Configuration of E2E Message 53
Figure 3-4 Imported Attributes in System Extract File. 54
Figure 3-5 AR Configuration for SignalGroup with E2E Configuration 55
Figure 3-6 E2EXf Module in DaVinci Configurator Classic and SomeIpXf Module in DaVinci Configurator Classic 56
Figure 3-7 Enable Data Transformation in DaVinci Developer Classic Workspace 56
Figure 3-8 Network Signal Mapping in DaVinci Developer Classic Workspace 57
Figure 3-9 Configuring the Transformation Parameters 57
Figure 3-10 Port Prototype and Use End-to-End Protection 58
Figure 3-11 Code Example of Generated Template Code 59
Figure 3-12 Code Example of Generated Template Code (Rte_write of Swc) 59
Figure 3-13 Code Example of Generated Template Code (Rte_write of OsApplication) 59
Figure 3-14 Code Example of Generated Template Code (ComXf Function) 60
Figure 3-15 Code Example of Generated Template Code (ComXf Function Implementation) 60
Figure 3-16 Defined Sub-Functions Inside of E2EXf 60
Figure 3-17 <SWC_Template>.c 61
Figure 3-18 Rte_<SWC>.h 61
Figure 3-19 Rte_<OsApplication>.c 61
Figure 3-20 E2EXf_LCfg.h 62
Figure 3-21 ComXf.h 62
Figure 3-22 ComXf.c 62
Figure 3-23 Schematic Overview 63
Figure 3-24 API Table Technical Reference RTE 64
Figure 3-25 Transformation Error Handling in DaVinci Developer Classix 64
Figure 3-26 Generated Code 65

Tables

Table 2-1 JavaScript Parameter 26
Table 2-2 JavaScript Parameter 32
Table 3-1 DBC Attributes Description in Accordance with System Description Extract for Profiles 54

1 Introduction and Explanation About this Document

This documentation explains how to generate the configuration files and code for E2E protection. This means the usage of

- E2E protection wrapper (E2ePw) and
- the E2E transformer solution (E2EXf).

First it is shown how to create the E2E configuration for an existing DaVinci Configurator Classic project using DaVinci Developer Classic. Each step in the DaVinci Developer is explained until the final export of the ARXML description file. Then this ARXML file will be imported into an existing DaVinci Configurator Classic project.



Note
The ARXML file does not necessarily have to be created by DaVinvi Developer Classic. It can also be generated by 3rd party tools if fully compatible.

Finally, the AUTOSAR elements for E2E protection wrapper is described. As basis for the description, the AUTOSAR System Template (version 4.2.2) is used. Workarounds are described to support features like variant handling as well as the use of different file formats with the E2E Protection Wrapper solution.

The next part of this document explains the integration of E2E Transformer. It is also separated into:

- configuration and
- code generation.

Strategies for error handling are described to make use of the return values obtained from the E2E protection state machine.



Note
Both AUTOSAR and Vector recommend the use of the E2E Transformer whenever possible and regard the E2E Protection Wrapper as a legacy solution. Only profiles 1 and 2 are implemented in the E2E Protection Wrapper and recent changes in the E2E specification, e.g. state machine additions in AUTOSAR R19-11 and R20-11, are not and will not be supported by the E2E Protection Wrapper.

2 E2E Protection Wrapper Integration

2.1 Configuration of E2E Protection in DaVinci Developer Classic

2.1.1 Create DaVinci Developer Classic Workspace Based on your ECUExtract

Note
This step can be skipped in case you already have a DaVinci Developer Workspace.

Open your DaVinci Developer Classic Workspace, select **File|New Workspace** and select your AUTOSAR version, required by your customer (e.g. ASR4.5.0). Then a DaVinci Developer Classic workspace is created.



Figure 2-1 Create New Workspace in DaVinci Developer Classic and Select AR Version of Workspace Files

Before you import your ECU Extract into DaVinci Developer Classic…
If there are some Short Names or other elements within your ECU Extract that are not conform to AUTOSAR, you need to rename them first to make the import work. Open the ECU Extract in e.g. Notepad++ and replace an incorrect Short Name using the following find and replace settings:

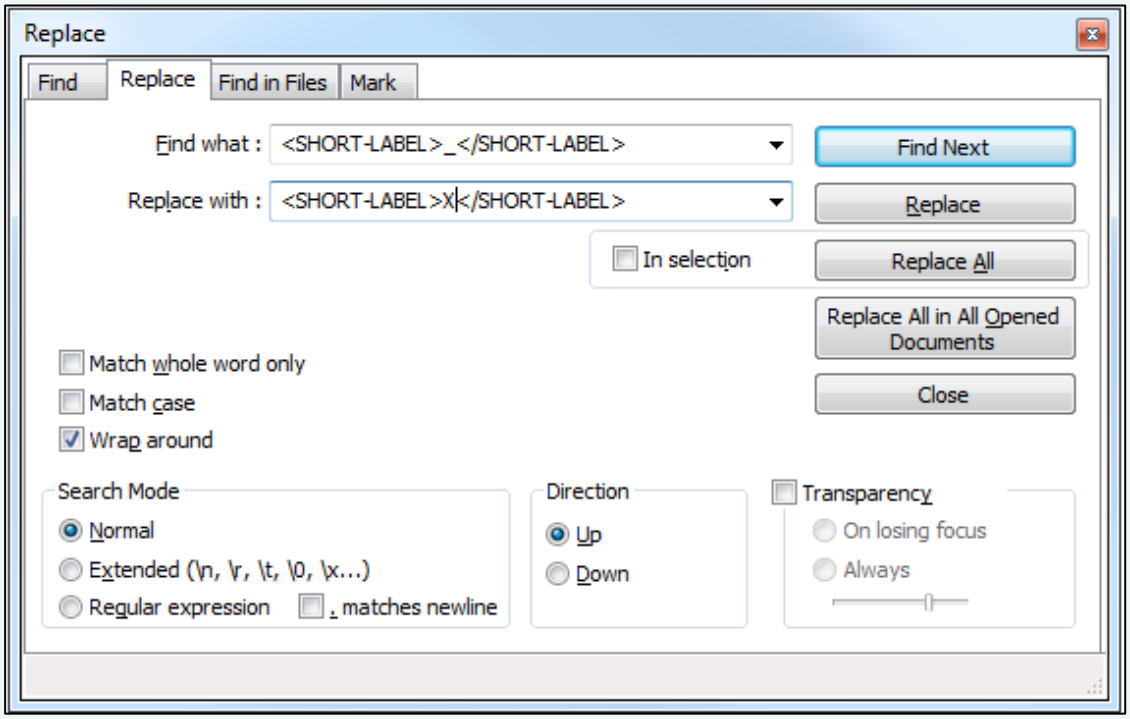


Figure 2-2 Modifying ECU Extract to Fulfill AR Scheme Requirements

Now import your ECU Extract into the DaVinci Developer Classic workspace, so that all the communication related information is present. This can be done via **Project|ARXML Import…**.

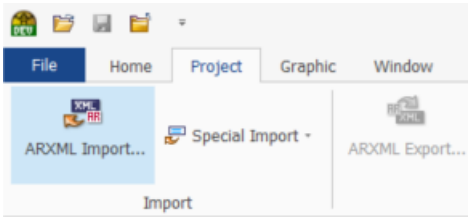


Figure 2-3 Import of ECU Extract in DaVinci Developer Classic Workspace

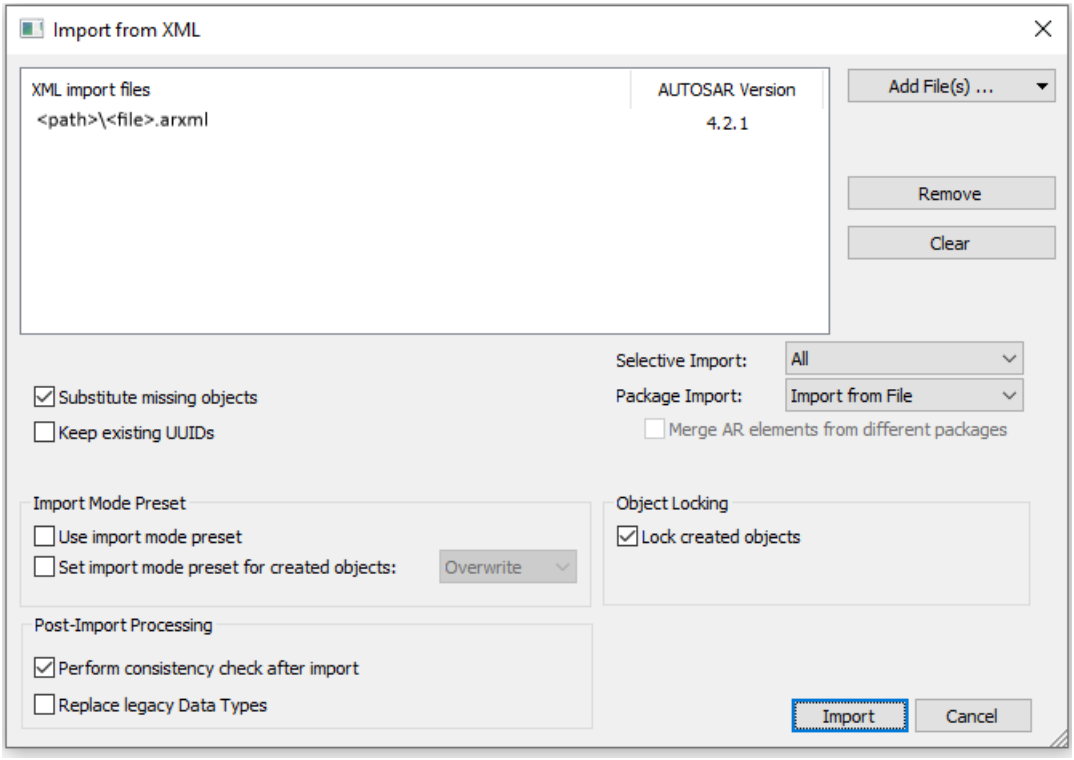


Figure 2-4 Import Dialog for Importing ECU Extract into DaVinci Developer Classic Workspace

After this import, the communication signals defined by OEM will be available within your DaVinci Developer Classic workspace. In the following you will use these signals to configure their E2E Protection.

2.1.2 Add a SWC and Port Prototype to the DaVinci Developer Classic Workspace

Before we can use the signals defined inside of your ECU Extract, we need to have a SWC, which uses these communication signals. It is **mandatory to have a Port-Access** of the E2e protected signal for the later generation step being successful. Therefore, go to the **Library** view and add a SWC. Right-click **Application Component Types** and select **New Application Component Type…**.

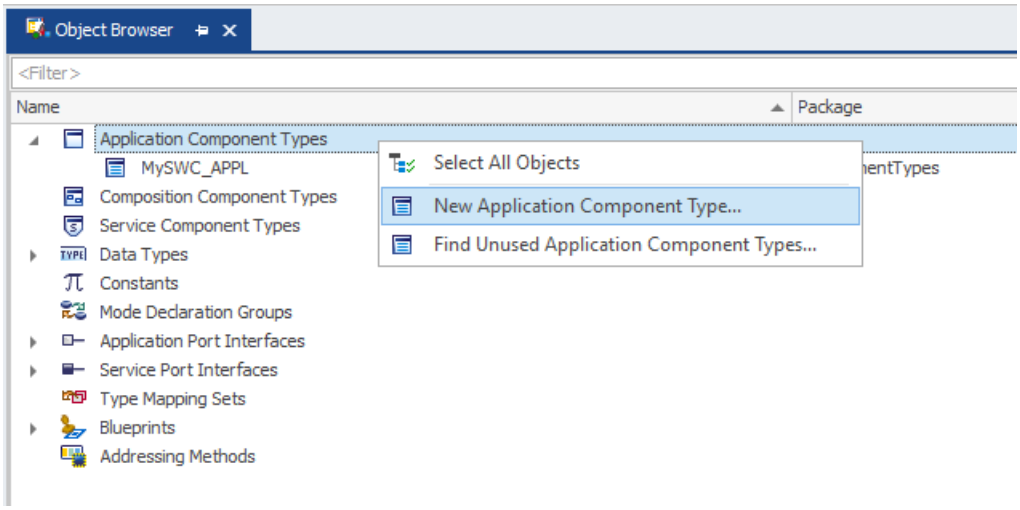


Figure 2-5 Adding Necessary Application Component Type in DaVinci Developer Classic

The Name of the **Component Type** can be chosen freely. Set **Atomic**, **Application** as **Type** and set the radio button to **Source code**. And confirm with **[OK]**.

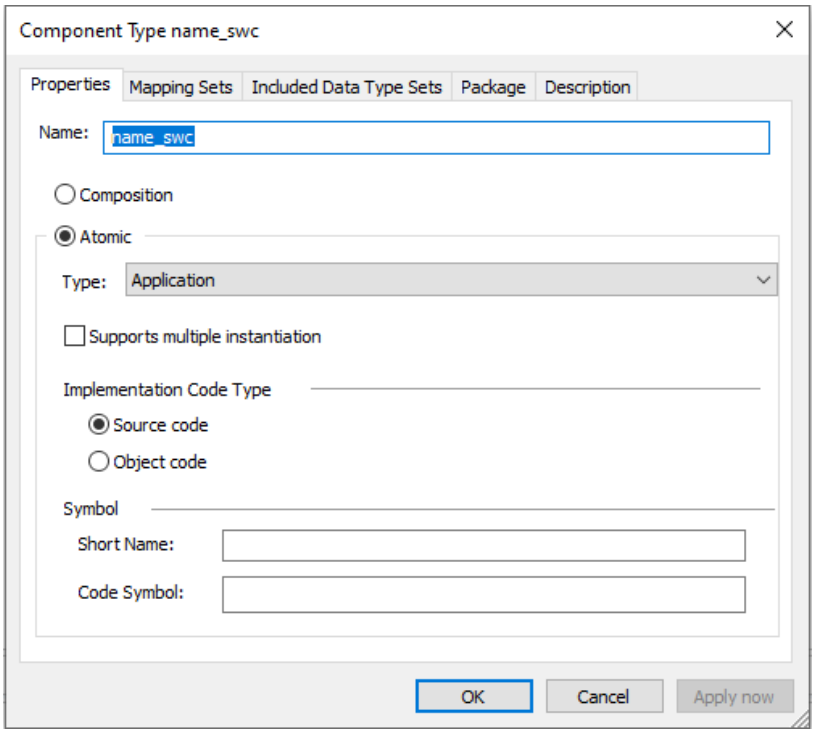


Figure 2-6 Configuring the Application Component

Switch to the Software Design view (double click).

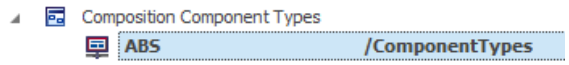


Figure 2-7 Selecting Software Design in DaVinci Developer Classic

Drag and drop the SWC from the **Library** view into the **Software Design** view:

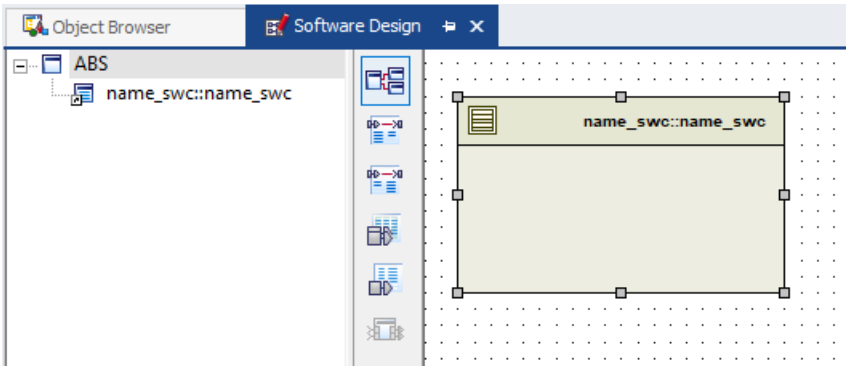


Figure 2-8 Creating the SWC Prototype in DaVinci Developer Classic

Double-click on the SWC, select the runnable view:

1. Click **[New]** in the **Runnable Entity List** and select **Runnable**
2. Switch to the **Trigger** view

3. Click **[New]** and select **Periodic…** to open the window **New Trigger Definition: Periodic**
4. Set the **Period** to **10 msec**, select **Automatic creation** and the Trigger will be created.

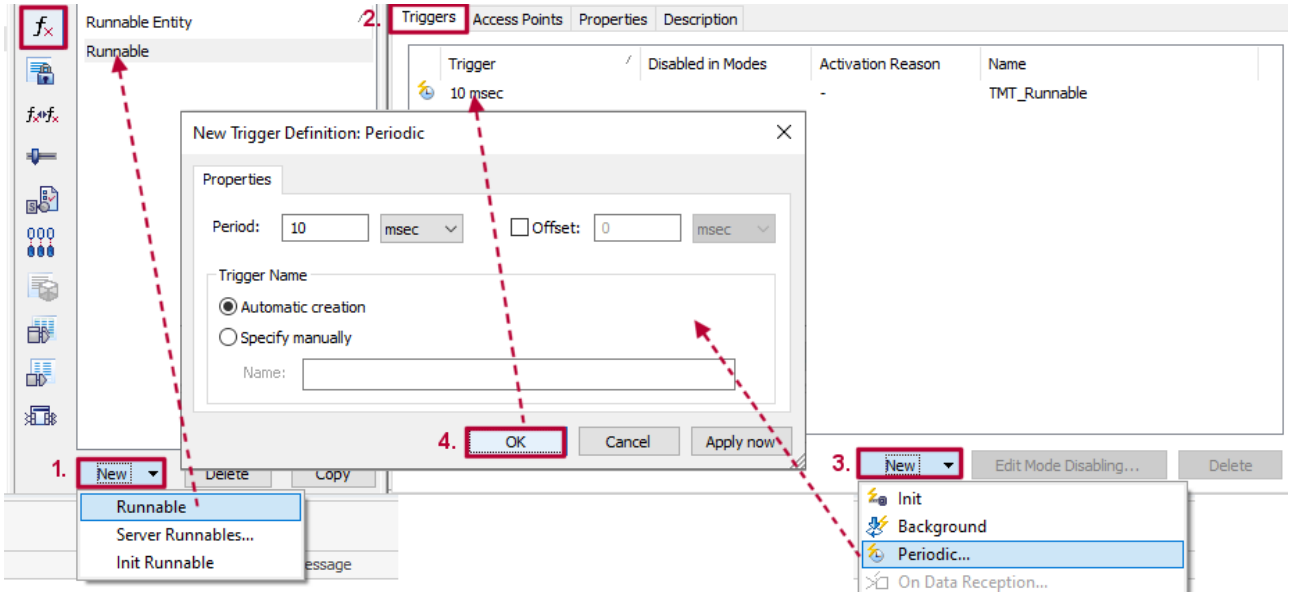


Figure 2-9 Runnable and Trigger for The New SWC

Now the dummy SWC for your DaVinci Developer Classic workspace is ready.

Connect the Dummy SWC to the Communication Signals

Therefore, go to the **Data Mapping** view.

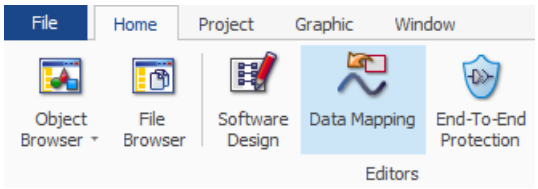


Figure 2-10 Switching to Data Mapping in DaVinci Developer Classic Workspace

1. Change to the **Signal View**
2. Select the E2E protected network signal, which shall be used by your application. Right-click on it and select **Create Port Prototypes…**
3. Create **Delegation Ports** to map network signals

Caution
Signal Group is mandatory for step 2.

Now the communication signals are accessible for your application.

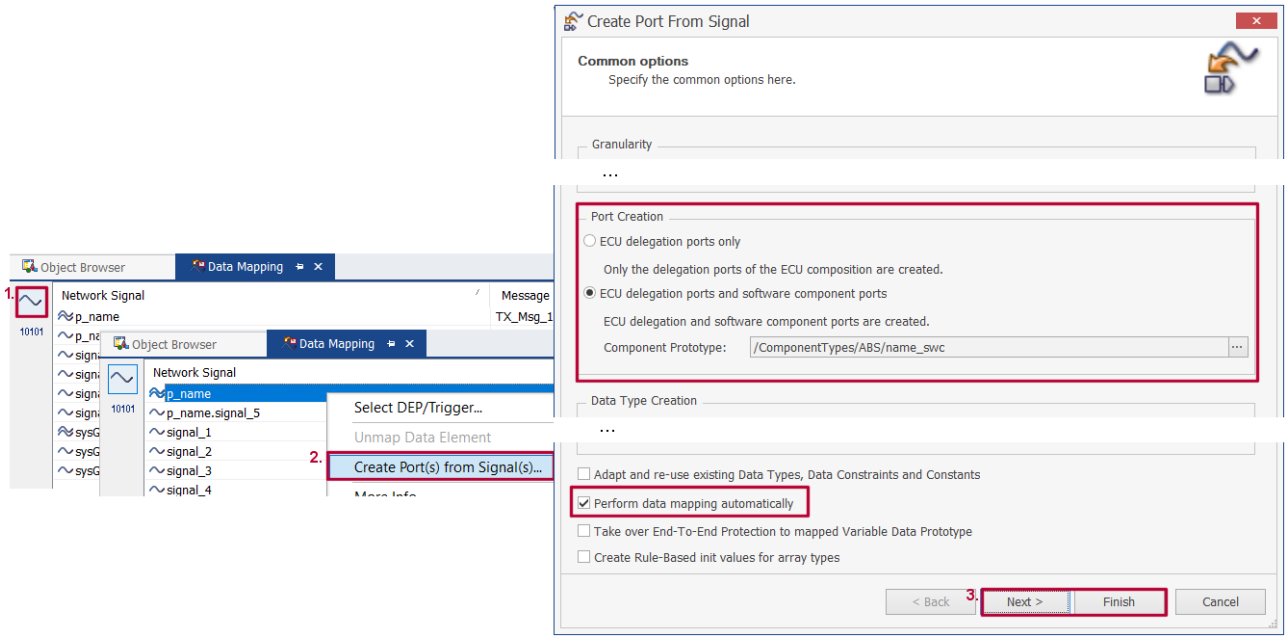


Figure 2-11 Network Signals, Mapping Network Signals to SWC Type,Creating Delegation Ports to Map Network Signals

One last thing concerning the dummy application is missing. You need to define a **Port Access**.

Go back to Software Design, double click on the SWC and change to the **Port Access** view. Click **[New]** and select a **Port Access** for all the available ports.

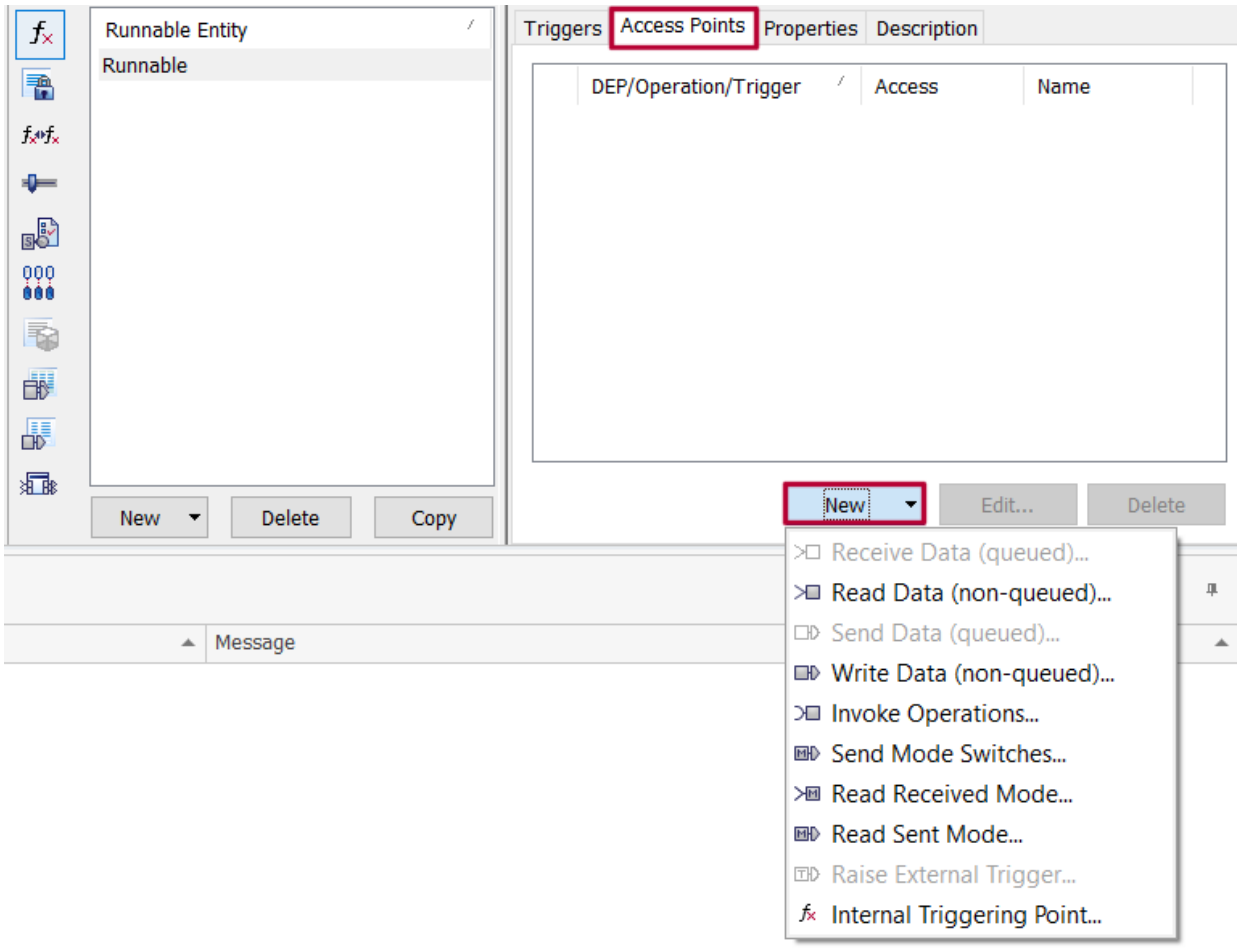


Figure 2-12 Add Port Access to Previously Created Runnable

In the Software Design now you will see that new ports are added to your software design.

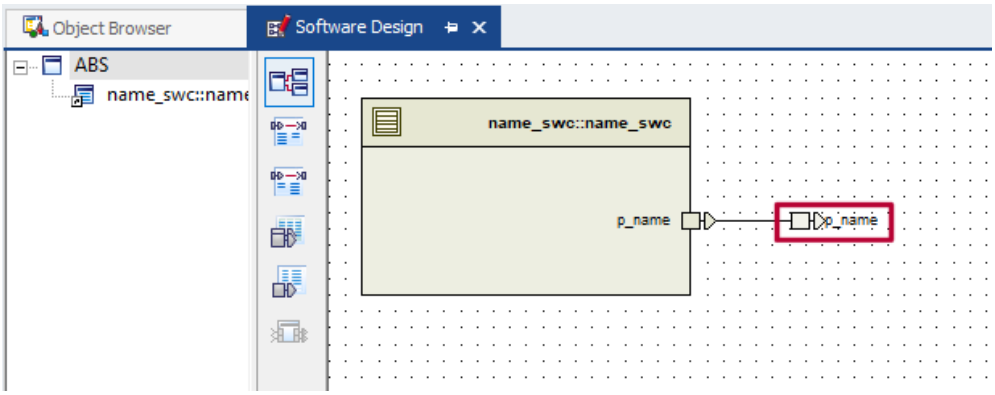


Figure 2-13 Created Delegation Port for Mapping of Network Signal

2.1.3 Configure E2E Configuration within the DaVinci Developer Classic

Prerequisites for the upcoming steps, which you should double-check:

> Having a DaVinci Developer Classic workspace with a SWC that should communicate via E2e

> Signal mapping (of Signal Group) to this SWC with a compatible Port Prototype

> Data Element of the Port Prototype has a configured port access in one of the runnables (read or write)

For configuring the **E2E protection** double-click on the delegation port, select the tag Communication Spec and activate the checkbox **Uses End to End Protection** feature for the port at the communication spec tab and confirm with **[OK]**.

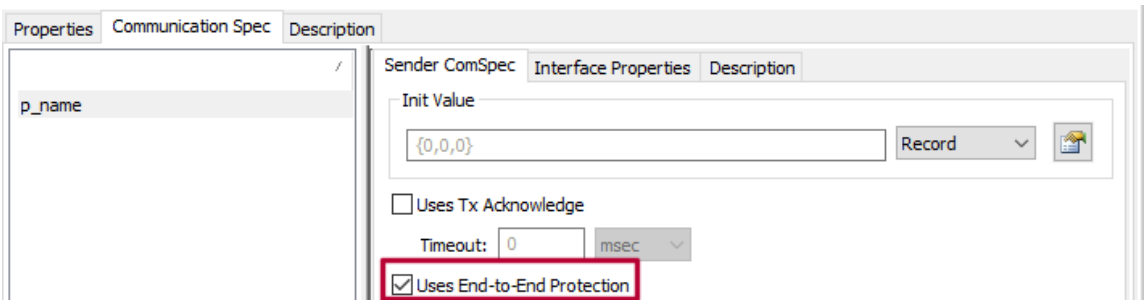


Figure 2-14 Activating End-To-End Protection for the Port Prototype

Now right-click on the port and select **Adapt Connected Port Prototypes…** and the window **Port Prototype Adaptations** will open. Choose the same settings as shown.

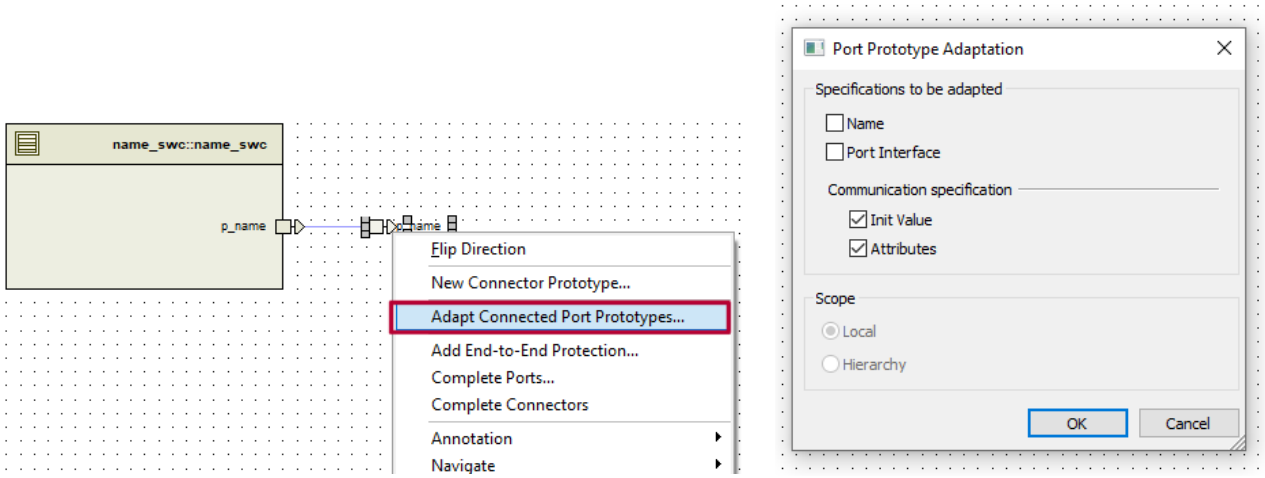


Figure 2-15 Adapt Connected Port Prototypes and Set Port Prototype Adaptations.

Now right click again on the port and choose **Add End-to-End Protection…**. Select the **Data Element Prototype** and confirm with **[OK]**. The window **End-to-End Protection Configuration** will open.

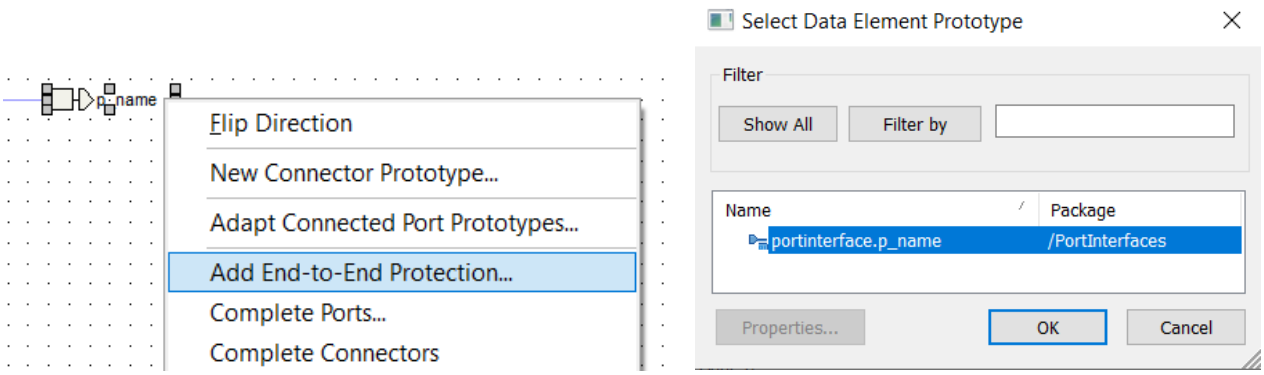


Figure 2-16 Select Element for E2E Configuration and Add E2E Configuration to Delegation Port

Configure the End-To-End-Protection in a way as required by the OEM.

Reference
Please have a look at the **UserManual_E2EPW.pdf**, which is part of your delivery for further information about the OEM-settings.

Example
This is just an example, how an OEM-specific setting could look like:

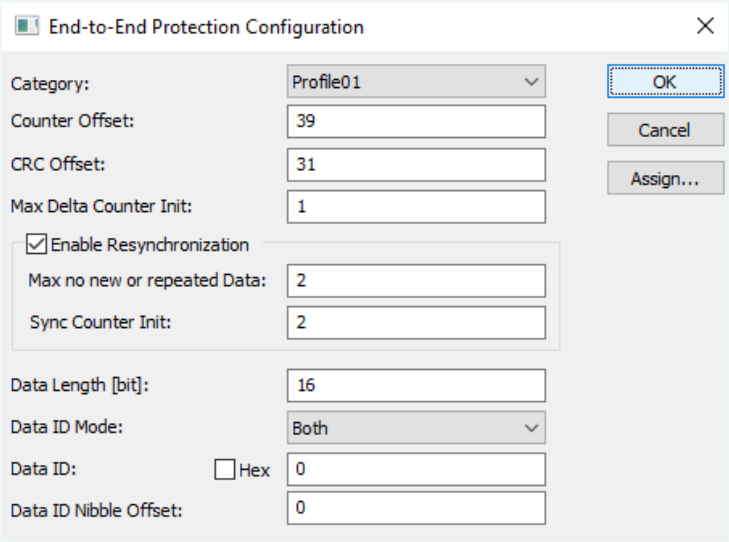


Figure 2-17 Example Configuration to Enable E2E Protection

Example for NOT using E2E protection

If you do NOT want to disable the E2ePw, choose the settings as shown here:

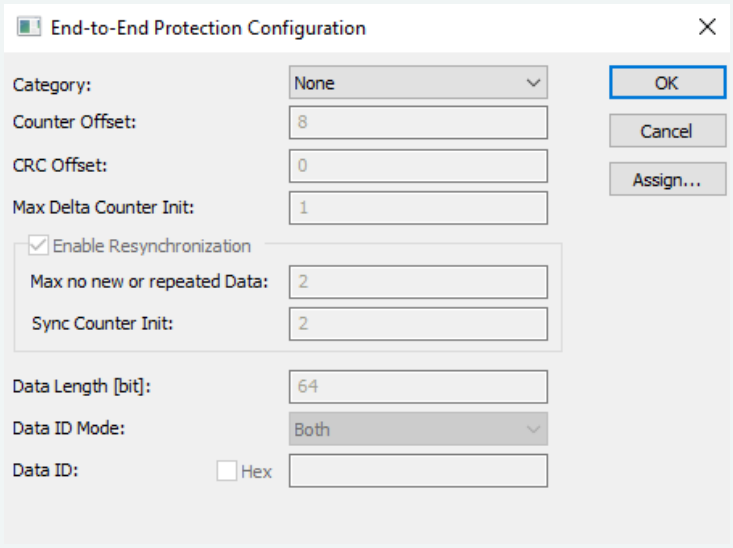


Figure 2-18 E2E Parameters to NOT Have E2ePw Enabled

2.1.4 Importing E2e Configuration to the MICROSAR Project (DaVinci Developer Classic Workspace is Connected to DaVinci Configurator Classic Project)

Prerequisites for the upcoming steps, which you should double-check:

> The DaVinci Developer Classic workspace gets synchronized to your DaVinci Configurator Classic project

The information, which was added above, can be synchronized into your DaVinci Configurator Classic project automatically by just saving the DaVinci Developer Classic workspace (and starting the synchronization). Both tools can exchange their configuration using the Ecu Flat-Extract, as depicted in the below illustration. No further manual steps are needed now.

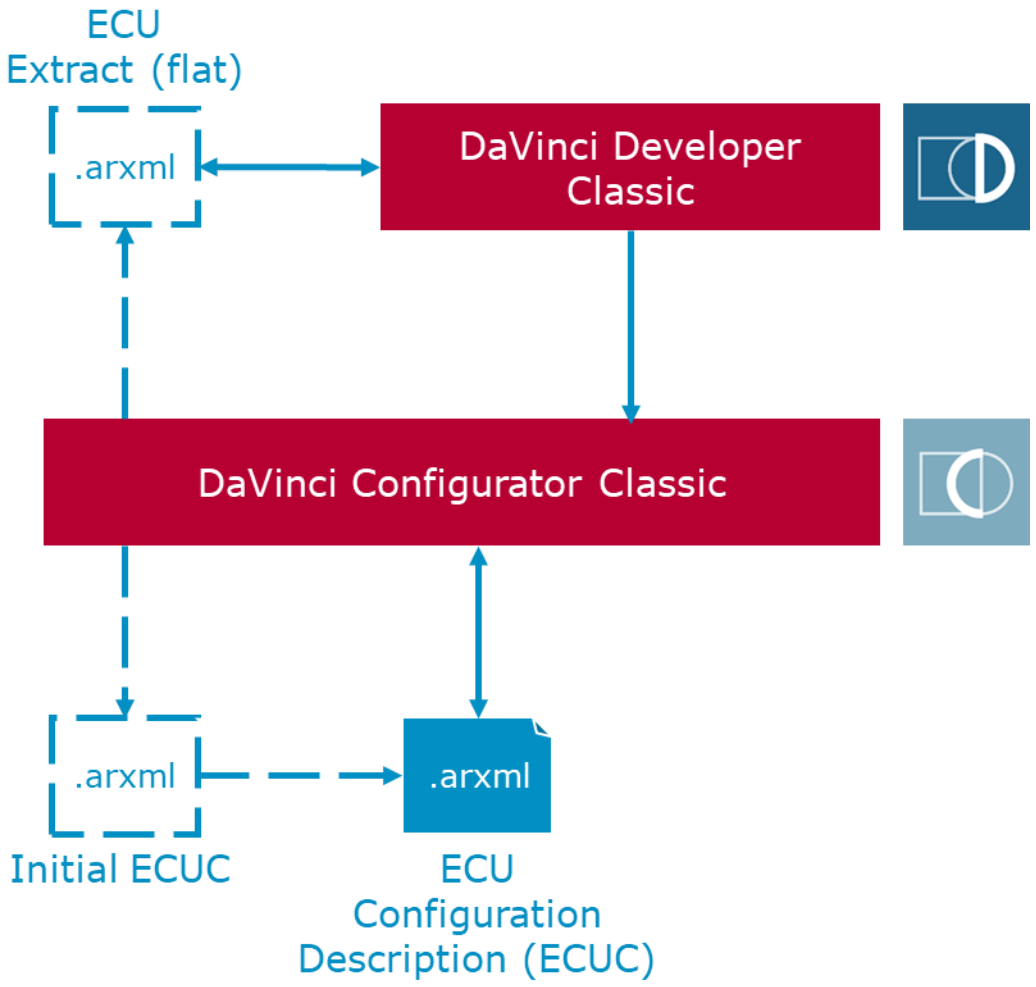


Figure 2-19 Illustration Showing How DaVinci Developer Classic and DaVinci Configurator Classic are Communicating With Each Other

2.1.5 Optional: Exporting E2e Configuration from DaVinci Developer Classic (e.g. to import it to DaVinci Configurator Classic Project)

After configuring all your needed End-To-End Protected Signal groups according to the requirements, you have to **export** this information to make it accessible by the DaVinci Configurator Classic. This can be done via **XML Export…** on the ECU Top Level Project.

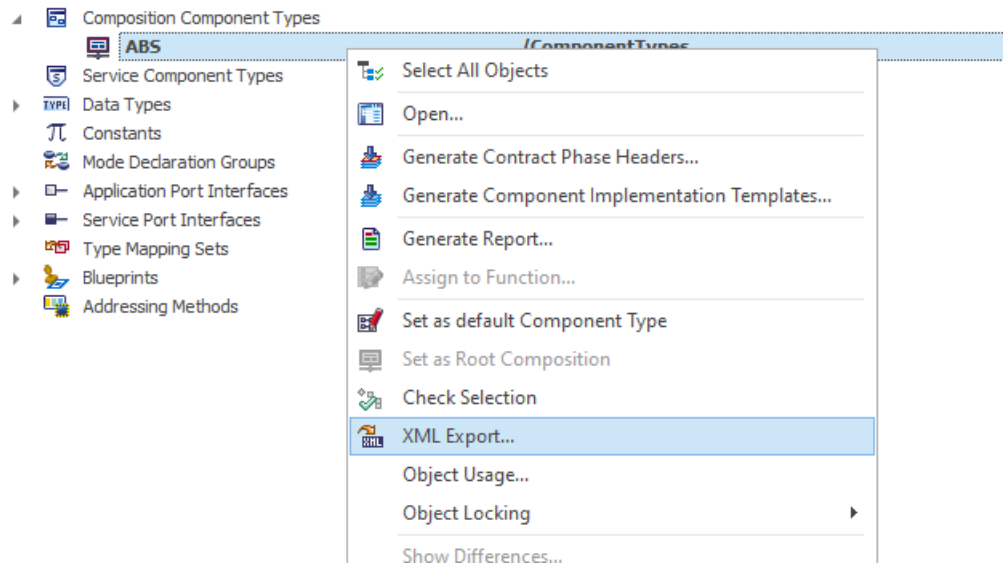


Figure 2-20 Exporting Configuration for Usage in DaVinci Configurator Classic Project

In the Export XML window, choose the settings as shown the following screenshot. This file is named <name>_SysDesc.

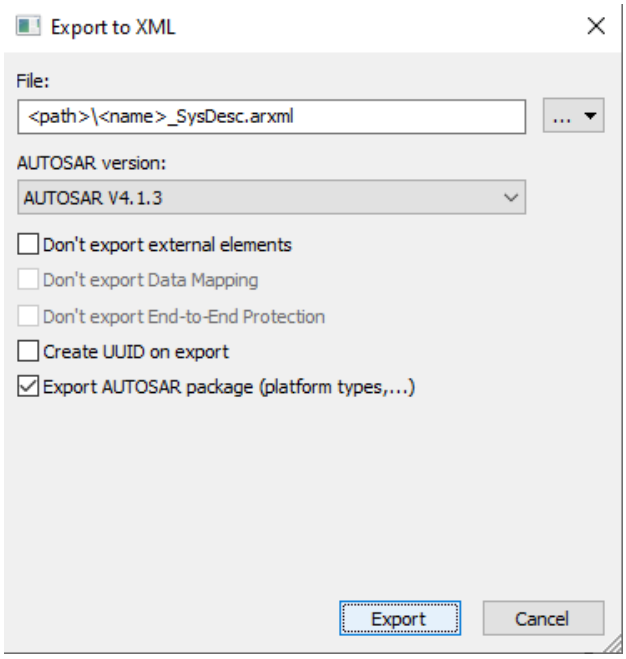


Figure 2-21 XML Export Dialog With Selection of AR Version

Now save the DaVinci Developer Classic Workspace (the tool is no longer needed for this process)

You can use the exported arxml to import your E2e Information into DaVinci Configurator Classic, which can execute the E2ePw code generation step. Alternatively, the information can be used to call the E2ePw (Pre-processor and Code-Generator) separated from any other tools (as described in [6]).

Open DaVinci Configurator Classic. Open the **Input Files Dialog** and add the **TopLevel Export** as an Input File.

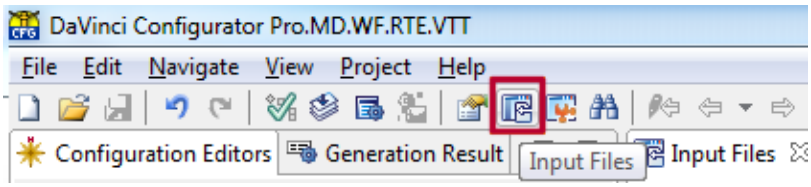


Figure 2-22 Open Input Files of Your DaVinci Configurator Classic Project

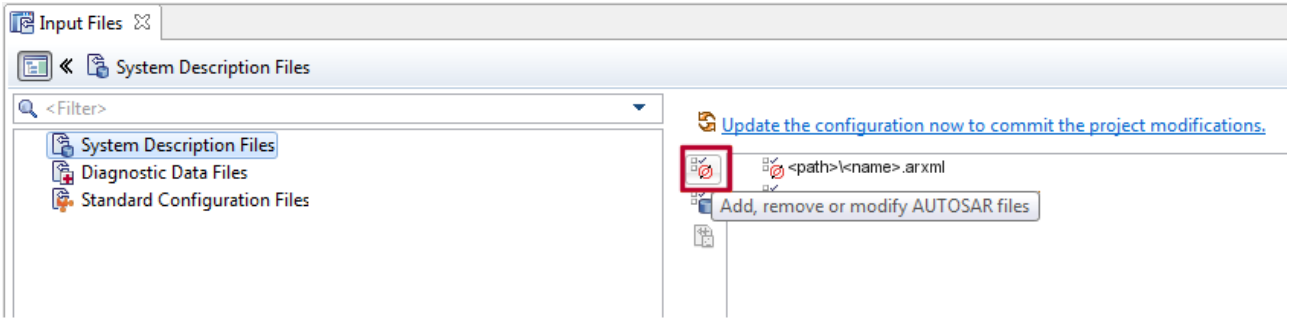


Figure 2-23 Add the DaVinci Developer Classic Export to Your DaVinci Configurator Classic Input System Description Files

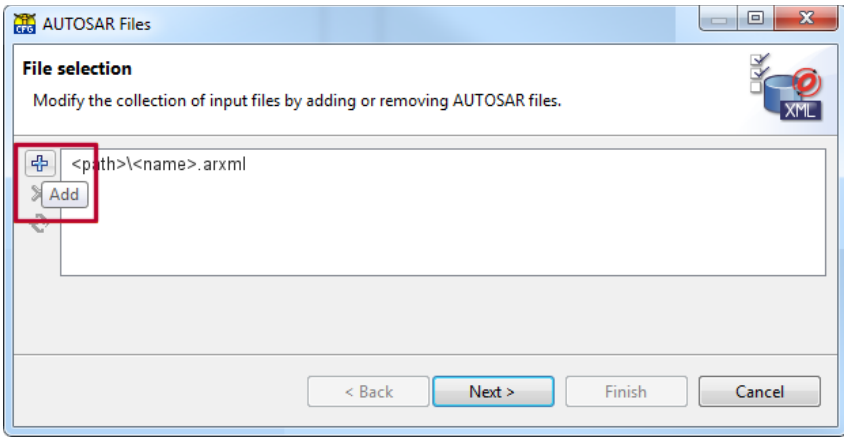


Figure 2-24 Select the Previously Exported File

It is important to **update** your configuration.

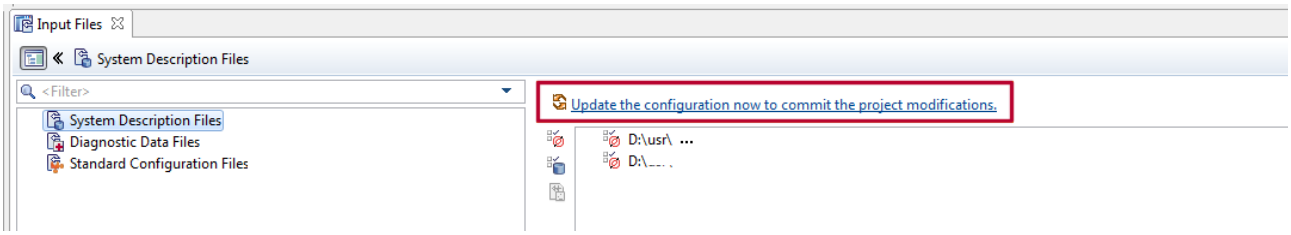


Figure 2-25 Update Your Configuration in DaVinci Configurator Classic

2.2 Generating the E2E Configuration and the Code in DaVinci Configurator Classic

The E2ePw generation step is integrated – in form of a generator - in the Configurator 5 (section: Complex Driver). It is started within the code generation, such as other code generators.

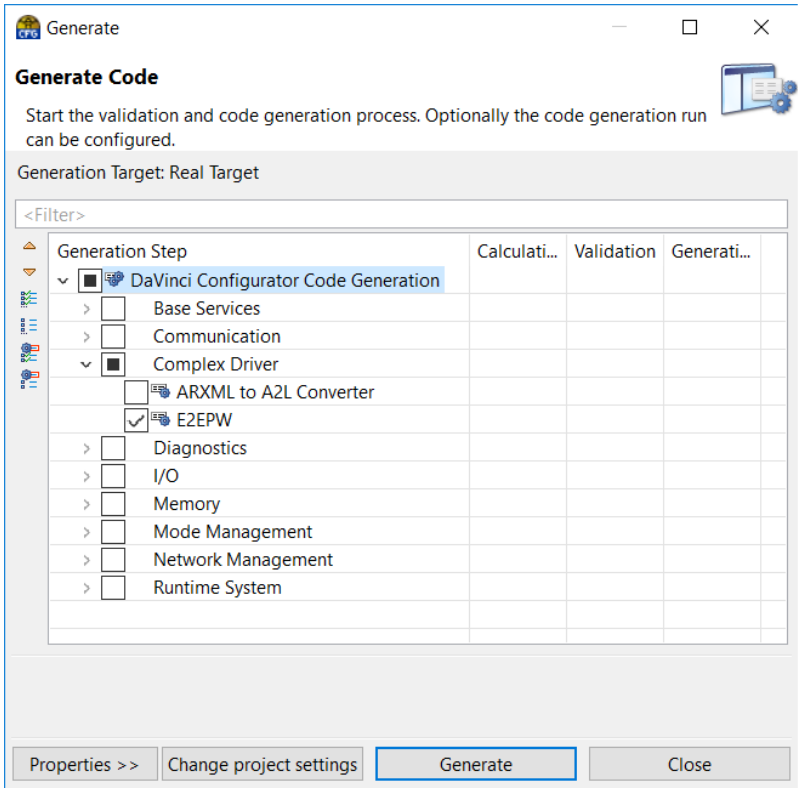


Figure 2-26 Generating the E2ePw Configuration and Code

Generally, the E2ePw generation is divided into two steps:

> Pre-processing of the ARXML file (and generation of a configuration file)

> Code generation (using the configuration from pre-processing)

Both steps can also be executed using the command line tools being part of your BSW Package. Further information about usage of the tool can be found in [6]. After the generator finished, this is indicated in green – such green symbol does not mean that the generation of code was successful, which is not intuitive on the first sight.

How else can you check successful execution?

You should enable the Generation Result view in DaVinci Configurator Classic, which shows the generated files. If there are no generated files, either the pre-processing or the code generation step did fail.

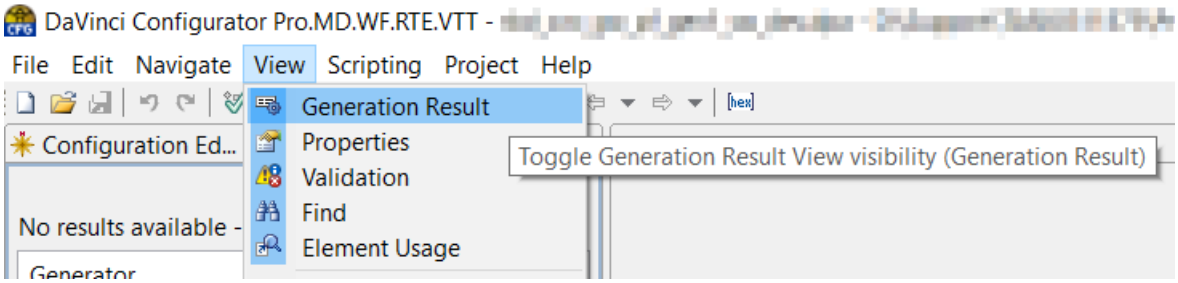


Figure 2-27 Enable Generation Results View in DaVinci Configurator Classic

Expectation: If the generation step worked fine, the pre-processor step creates the *.cfg file, which is post-processed to create the e2e_pw code in the second step (both steps are performed within the generator step…).

These files should be visible in the Generation Results view at the Generator E2ePw. Find below a screenshot for the generation result of AWDGp signal group, which was configured in the steps above

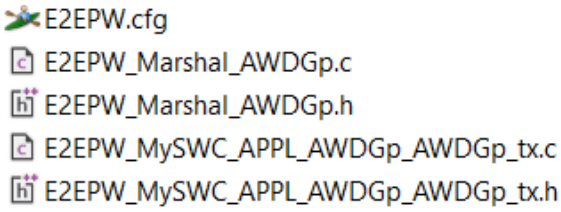


Figure 2-28 Results From Generation: Configuration File (E2ePw.cfg) and E2ePw Code Files

If these or not all expected files are not generated within your project, you find further information in chapter 2.3. First, you should find out which of the two steps fails and what is the reason.

2.3 Debugging in Case the Code Generation Step Was Not Executed Successfully

If one of the generation steps fails (the generator might still show a green tick), you get further information in the **Generation Results** view. Additionally, you should enable the console window to see the interaction between the E2ePw generation command line tools and the DaVinci Configurator Classic (see below screenshot about how to enable the console output window).

In the below screenshot you see that the preprocessor failed, due to missing parameters. This indicates that the E2e Protection Set was configured incorrectly in the first place, since the Max_Delta_Counter_Init is missing.

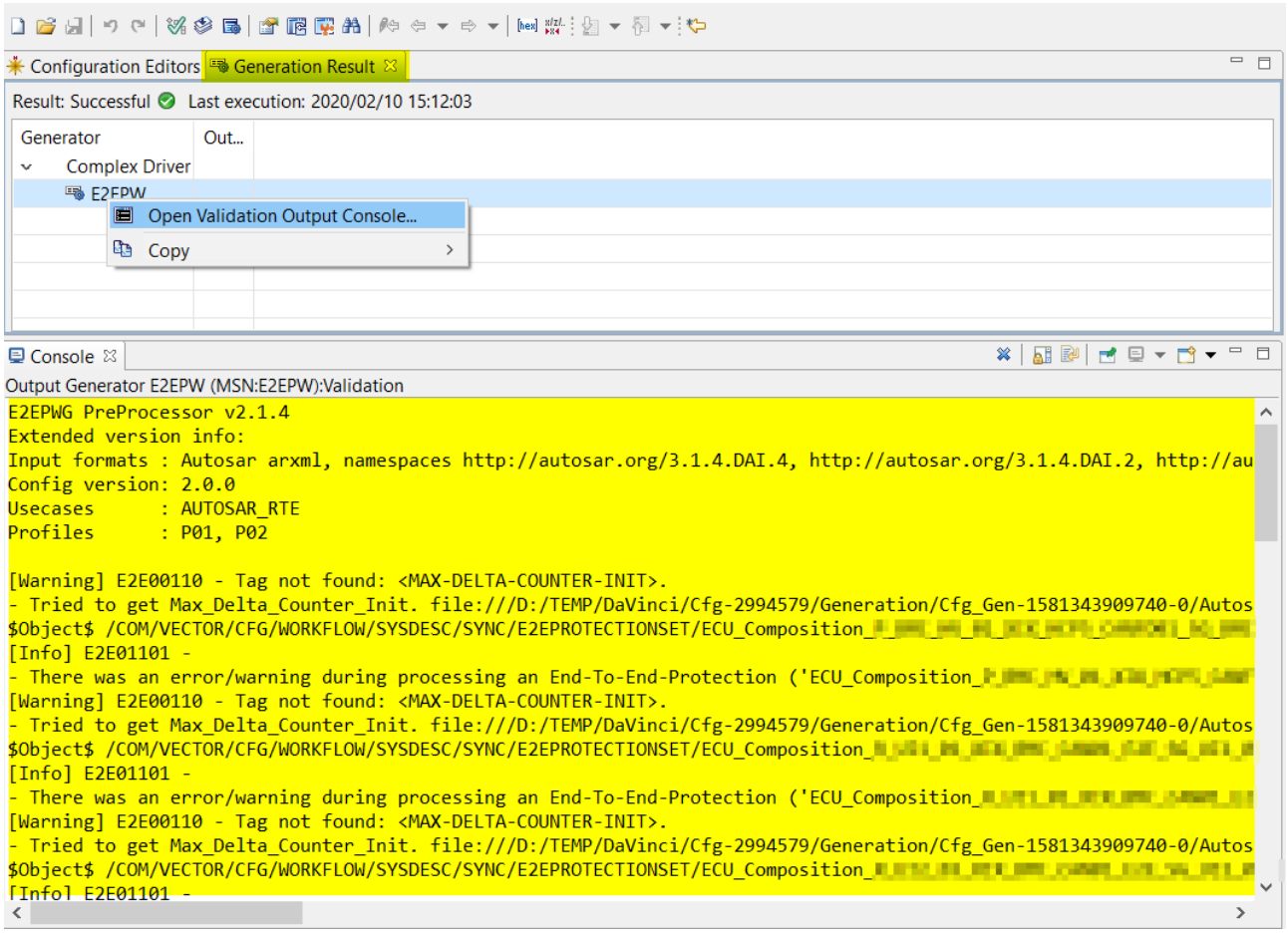


Figure 2-29 Console Output of the E2ePw Generation Step

If you are not sure about the error code, you should look it up in [6]. In case you cannot find any valuable information in there, contact the Vector Embedded Support and attach all error information that you have by now (especially the Console output of the generation step).

2.4 Using E2ePw With OEM ARXML File

The Vector Embedded Support created a Java script, which is modifying the OEM ARXML files depending on the use case. If you have not received the script, please request it (including the script's name) via e-mail to support@vector.com.

Then handling of both artifacts, is described step-by-step in the following subchapters.

2.4.1 Using E2ePw With Variants

This chapter explains how to generate the end-to-end protection wrappers (E2ePw) for projects that have multiple variants for E2e protected signal mappings. The complete workflow consists of following step:

> Use the JavaScript **AdaptSystemSignalCommunicationForE2EPW** to rename the variant Pdu mappings. (This step removes the variant from the mapping).

2.4.1.1 Use the JavaScript AdaptSystemSignalCommunicationForE2EPW

The described approach works if the protected signal groups are shared between the different variants.

In case there are signal groups shared between two or more variants, the generation of the E2E protection wrappers is still possible, if the layout of the signal group/message is the same for all variants. Otherwise, the script will rename the signal group/message with a suffix, which should be used for renaming the signals (e.g. the variant name).

2.4.1.2 Add the JavaScript to Your Project

To use the JavaScript “AdaptSystemSignalCommunicationForE2EPW” within your project, you need to add it to your project configuration first. Please enable the Script View and add the location of the script, as shown in the screenshot below.

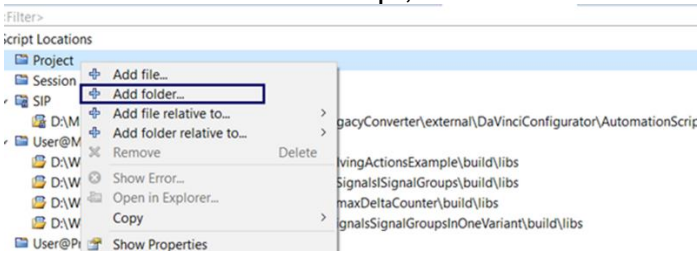


Figure 2-29 Add the JavaScript Location to Your Project Script Folders

Limitations:

You need a version 1.8.0 of AutomationInterface to run the script.



Figure 2-30 Configurator Issue

If you faced this issue, you should update your DaVinci Configurator Classic with the latest service pack.

2.4.1.3 Execute the JavaScript

After adding the script location to your configuration, you can execute the script by right-clicking on it. This step is shown in the screenshot below.

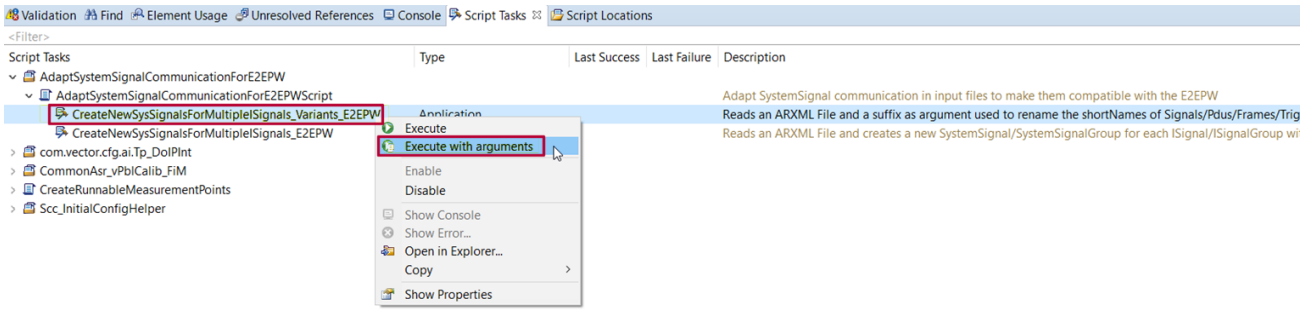


Figure 2-30 Executing the JavaScript " AdaptSystemSignalCommunicationForE2EPW "

After starting the JavaScript, it requests for four arguments. This request is shown in the below screenshot.

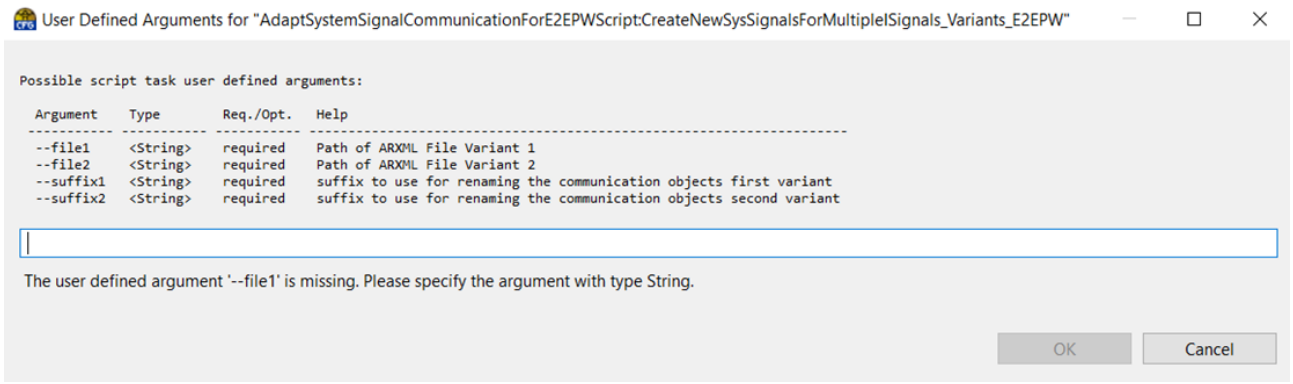


Figure 2-31 Arguments to Execute the JavaScript AdaptSystemSignalCommunicationForE2EPW

Please find below further details about the four parameters of the JavaScript.

Attribute	Description
--file1	Path to the ARXML file to variant 1
--file2	Path to the ARXML file to variant 2
--suffix1	Suffix to use for renaming the communication objects first variant
--suffix2	Suffix to use for renaming the communication objects second variant

Table 2-1 JavaScript Parameter

Below you find an example to feed the script with attributes:

Note
no line-break or linefeed should be used.

--file D:\Work\JavaSkripte\RenameSignals\<OEM>_L.arxml **--file** D:\Work\JavaSkripte\RenameSignals\<OEM>_R.arxml **--suffix1** suffixL **--suffix2** suffixR

The script compares the referred files in the command line and checks the layout of identical Pdus in both ARXML files. The identical **ISignalGroups** and **ISignals** with a different layout and the associated **SystemSignalGroups/SystemSignals** are renamed to **Original Name + Suffix.**

2.4.1.4 Results of Executing the JavaScript

The changes are then saved in a new file with the name ***patched**. The patched output file get located to the same folder as the original (input) file.



Figure 2-323 Output File and Original File After Using the Script

If you have a look in the DaVinci Developer Classic, you can see the modification. The suffix, which was defined in suffix command line, is added as described before.



Figure 2-34 Updates in the ARXML Performed by the JavaScript

Finally, the above shown output file from the script should be used as new input file for updating the DaVinci Configurator Classic project. With other words: it has replaced the original input file.

2.4.1.5 Rework DaVinci Developer Classic Workspace

If you try to generate the E2ePw the following error will be reported:

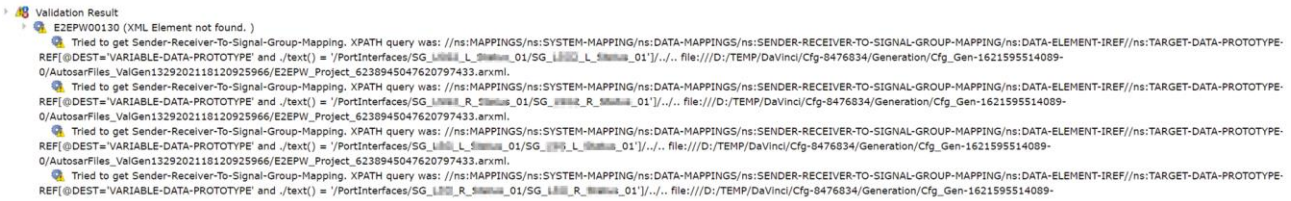


Figure 2-35 Error Reported from E2ePw

To work around this problem, you need to create a **Port** based on the **SystemSignalGroup** in DaVinci Developer Classic and to delete the **Predefined Variants**.

1. First check and delete the predefined Variants for **all** Signals.

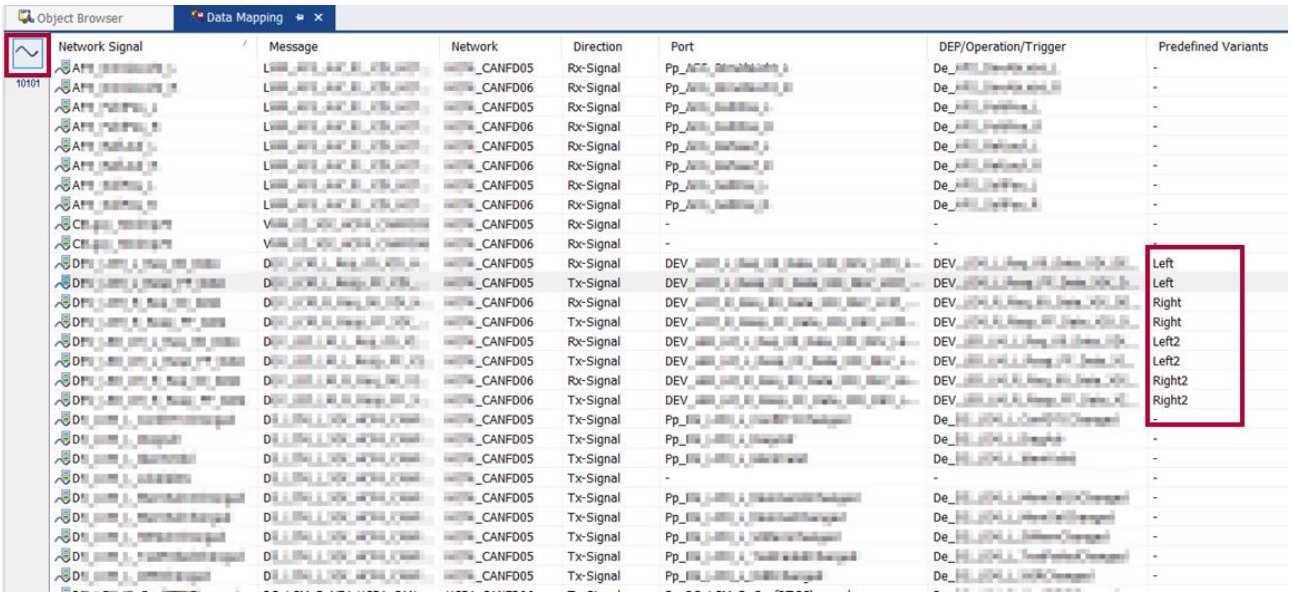


Figure 2-36 DaVinci Developer Classic Predefined Variants

If there are Predefined Values, then you must delete it by selecting the **<None>** option as shown:

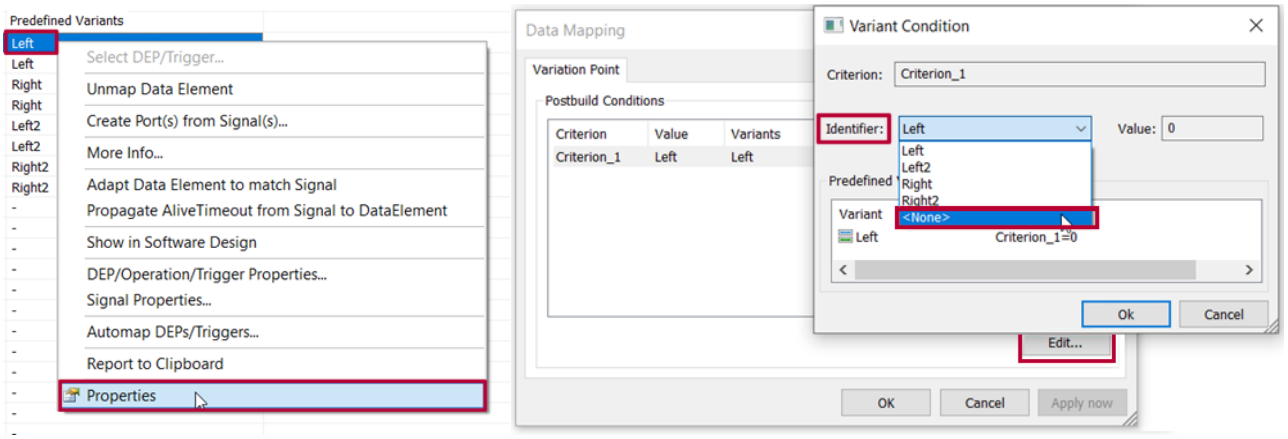


Figure 2-36 DaVinci Developer Classic Identifier Option

2. Second create a Port based on the SystemSignalGroup

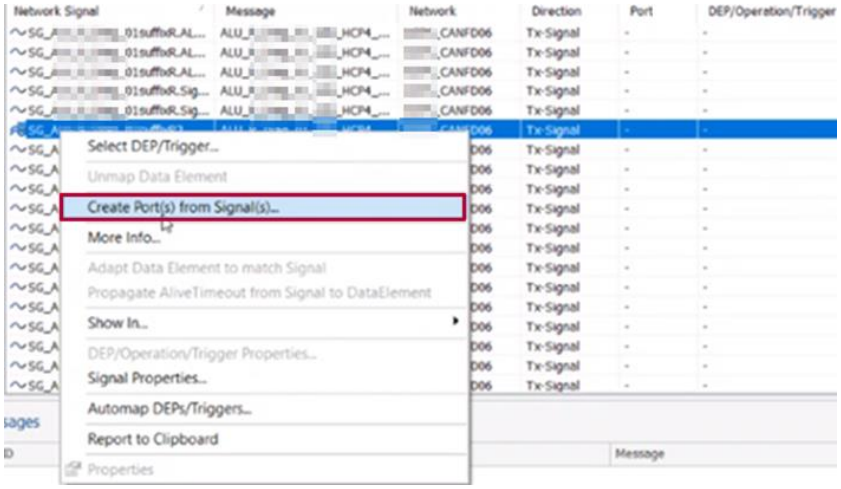


Figure 2-36 DaVinci Developer Classic Signals View

And create not just the delegation Port but also a Port App SWC Ports and enable the CheckBox **Take over EndToEndProtection to mapped VariableDataPrototype**.

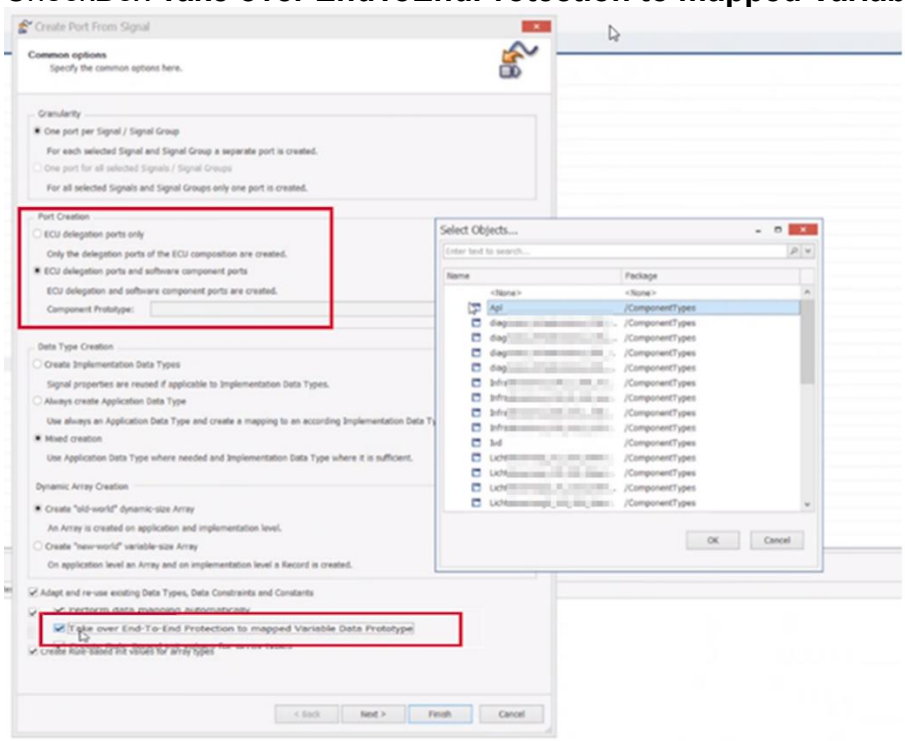


Figure 2-33 DaVinci Developer Classic Create Port From Signal View

Note
You are done now, but…if the project consists of multiple variants, then you need to repeat the steps for other input files. For example you might need to first patch the variants L1 and R1 and afterwards L2 and R2.

Finally, all variant data mappings should be removed (in each variant of your project).

2.4.2 Using E2ePw With FanIn/FanOuts ISignal Input File (Single Project)

This chapter explains how to generate the end-to-end protection wrappers (E2EPW) for projects without variants and in the ARXML input file(s) ISignals with a FanIn/FanOut configuration were modelled.

The complete workflow consists of following step:

> Use the JavaScript **AdaptSystemSignalCommunicationForE2EPW** to rename the variant Pdu mappings. (This step removes the variant from the mapping).

2.4.2.1 What Means FanIn/FanOuts ISignals

There are two types of FanIn/FanOut signals, an Rte and a Com SignalGateway FanIn/FanOut. In this case we describe an Rte FanIn/FanOut, an Rte feature which allows to send/receive one SystemSignal by several IPdus.
However this feature is not supported by the E2ePw. Therefore, the script dissolves this behavior by creating one SystemSignal/SystemSignalGroup for each referenced ISignal/ISignalGroup.

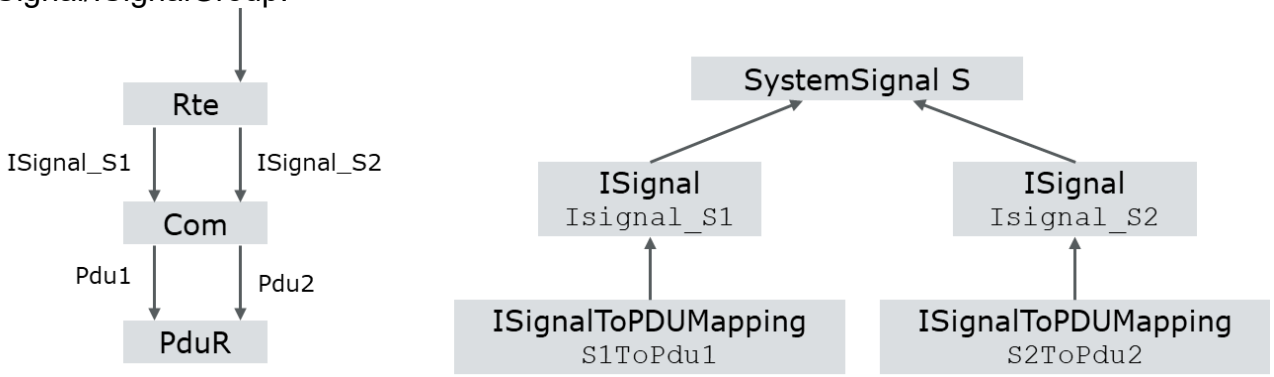


Figure 2-38 Rte Fan-Out From AUTOSAR_TPS_SystemTemplate.pdf

The picture right above shows an example of a SystemSignal which was modelled according to a FanIn/FanOut model. Two ISignals reference the same SystemSignal, the modulation in the input file looks like so:

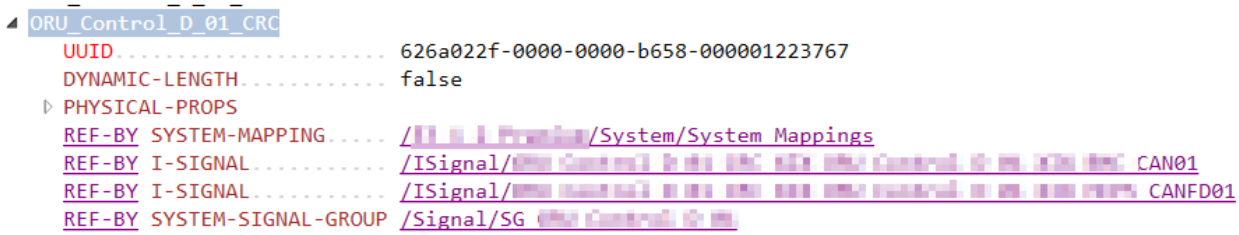


Figure 2-39 Rte Fan-Out From in ARXML File

xxx_CRC is mapped to one SystemSignalGroup and should be routed to different CAN bus.

Reference
More details can be found in AUTOSAR_TPS_SystemTemplate.pdf.

2.4.2.2 Use the JavaScript AdaptSystemSignalCommunicationForE2EPW

The described approach works if the protected signal groups are modulated as FanIn/Fanout.

In case the script will rename the signal group/message with a suffix, which should be used for renaming the signals.

2.4.2.3 Add the JavaScript to Your Project

To use the JavaScript AdaptSystemSignalCommunicationForE2EPW within your project, you need to add it to your project configuration first. Please enable the Script View and add the location of the script, as shown in the screenshot below:

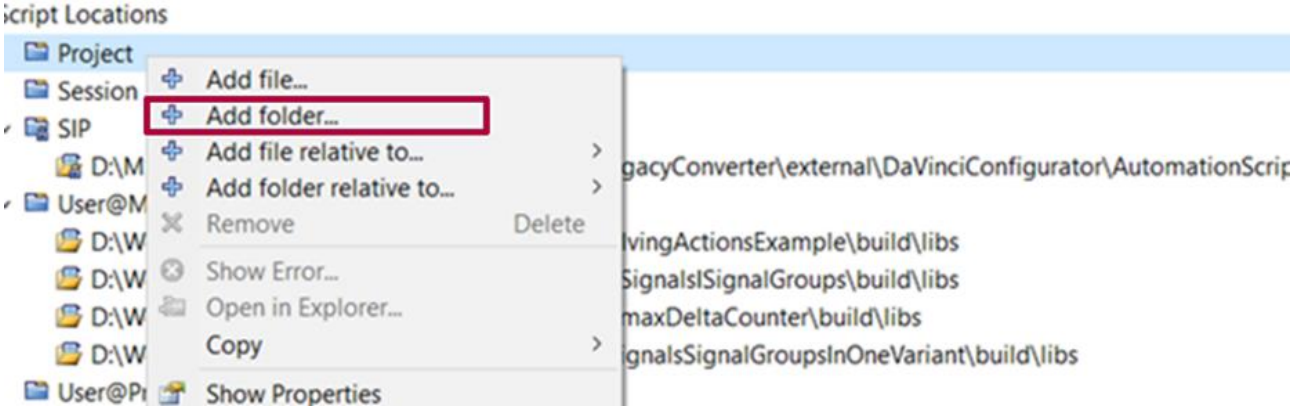


Figure 2-40 Add the JavaScript Location to Your Project Script Folders

Limitations:

You need a version 1.8.0 of AutomationInterface to run the script.

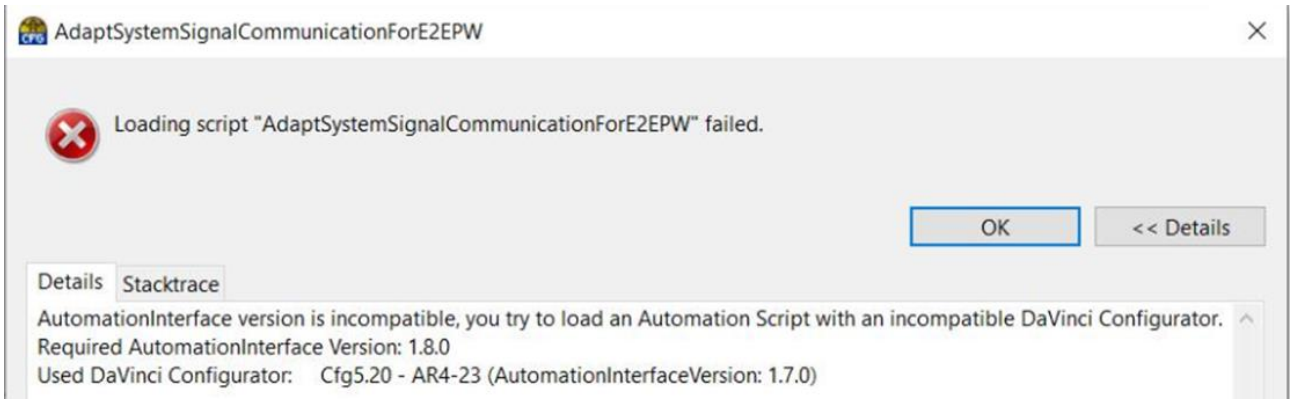


Figure 2-41 DaVinci Configurator Classic Issue

If you faced this issue, you should update your DaVinci Configurator Classic with the latest service pack.

2.4.2.4 Execute the JavaScript

After adding the script location to your configuration, you can execute the script by right-clicking on it. This step is show in the screenshot below.

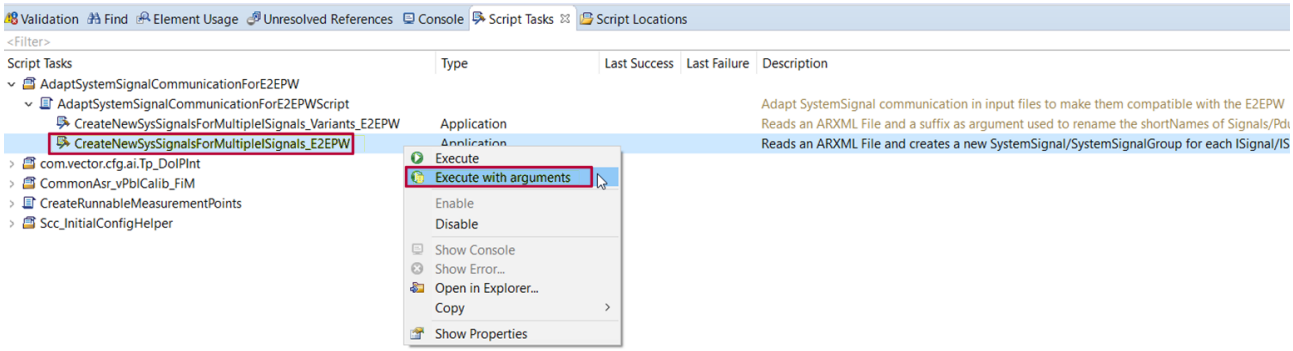


Figure 2-42 Executing the JavaScript AdaptSystemSignalCommunicationForE2EPW

After starting the JavaScript, it requests for one argument. This request is shown in the below screenshot.

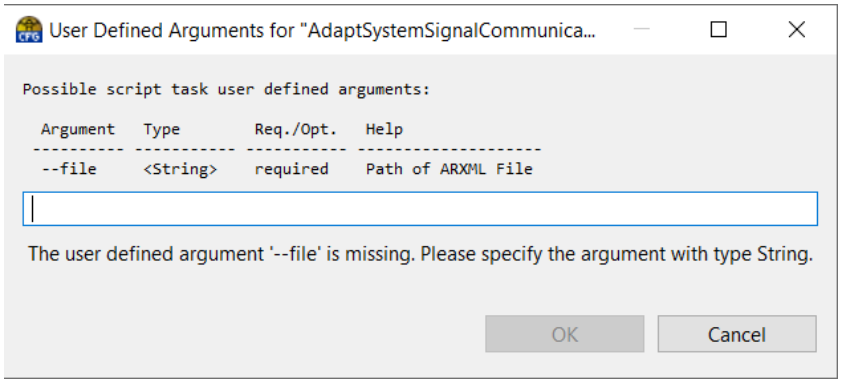


Figure 2-43 Arguments to Execute the JavaScript AdaptSystemSignalCommunicationForE2EPW

Please find below further details about the parameter of the JavaScript.

Attribure	Description
--file	Path to the ARXML

Table 2-2 JavaScript Parameter

Below you find an example to feed the script with attributes (Info: no line-break or linefeed should be used):

--file D:\Work\JavaSkripte\RenameSignals\<OEM>.arxml

The script search for FanIn/FanOut pdus in the ARXML file and renamed them.

2.4.2.5 Results of Excuting the JavaScript

The changes are then saved in a new file with the name ***patched**. The patched output file get located to the same folder as the original (input) file.



Figure 2-44 Output File and Original File After Using the Script

If you have a look in the DaVinci Developer Classic, you can see the modification. The Suffix, which was defined in suffix command line, is added as described before.

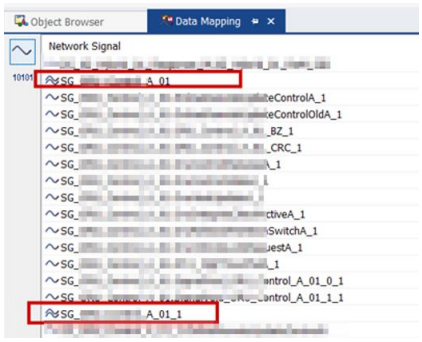


Figure 2-45 Updates in the ARXML Performed by the JavaScript

Finally, the above shown output file from the script should be used as new input file for updating the DaVinci Configurator Classic project. With other words: it has replaced the original input file.

2.4.2.6 Rework DaVinci Developer Classic Workspace

If you try to generate the E2ePw the following error will be reported:

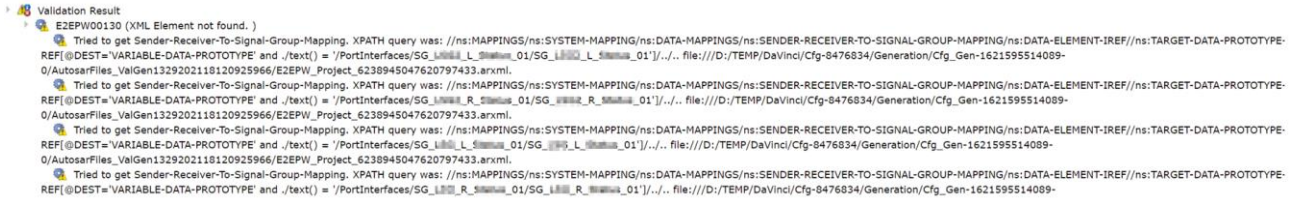


Figure 2-46 Error Reported From E2ePw

To work around this problem, you need to create a Port based on the SystemSignalGroup in DaVinci Developer Classic.

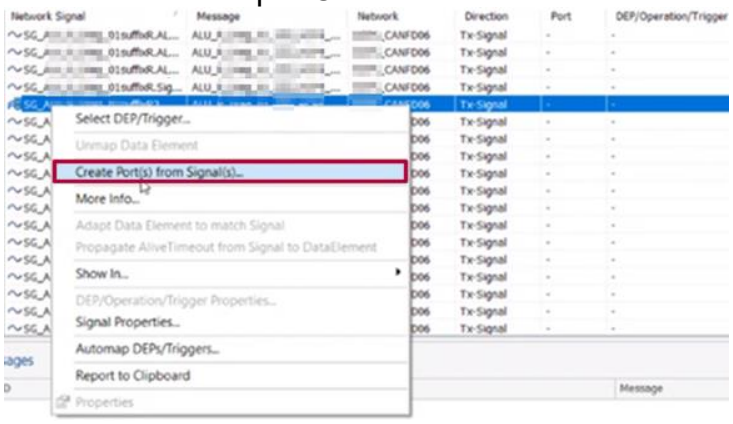


Figure 2-47 DaVinci Developer Classic Signals View

Create the delegation Port and also a port App SWC Ports and enable the check box **Take over EndToEndProtection to mapped VariableDataPrototype**.

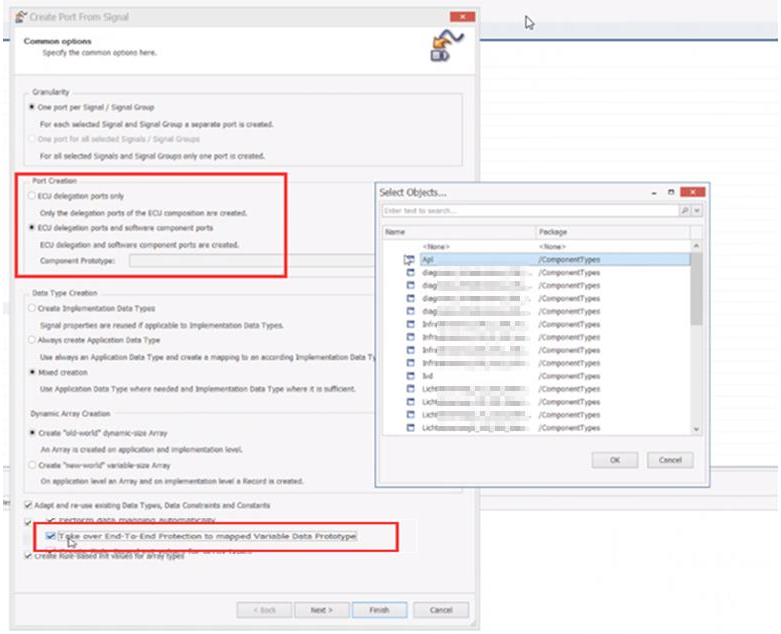


Figure 2-48 DaVinci Developer Create Port from Signal view

2.5 Using E2ePw With Variants in Signal Mapping

This chapter explains how to generate the end-to-end protection wrappers (E2EPW) for projects that have multiple variants for E2e protected signal mappings. The complete workflow consists of four steps:

1. Enable the end-to-end protection for a data mapping in the DaVinci Developer Classic
2. Create invariant system extracts for each of the four variants
3. Create configuration files for the E2EPW generator
4. Use the E2EPW generator to generate the source code for the E2EPW

Limitations:

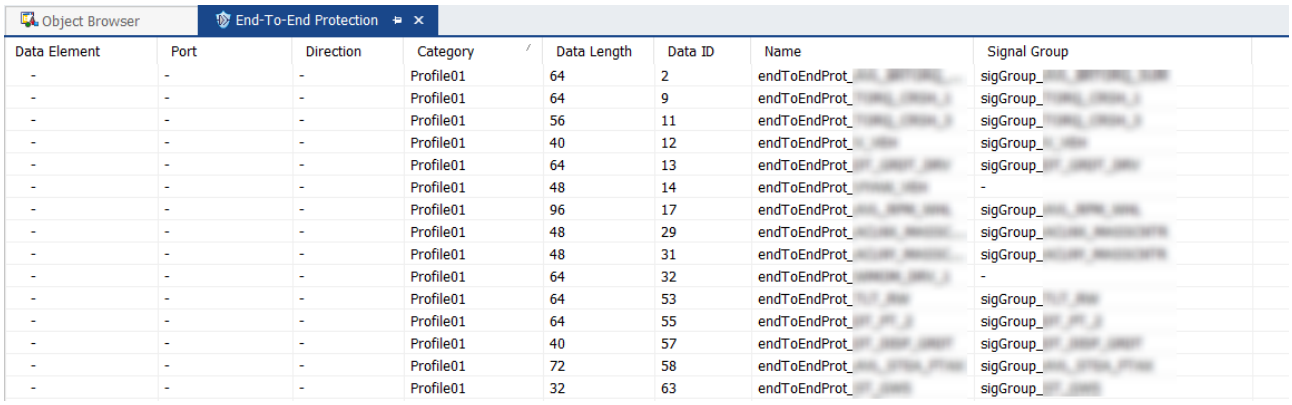
The described approach works only, if the protected signal groups are not shared between the different variants.

In case there are signal groups shared between two or more variants, the generation of the E2E protection wrappers is still possible, if the layout of the signal group/message is the same for all variants.

2.5.1 Step 1: Enable End-to-End Protection

You can enable the End-to-End (E2E) protection for each port with a data mapping using the DaVinci Developer Classic.

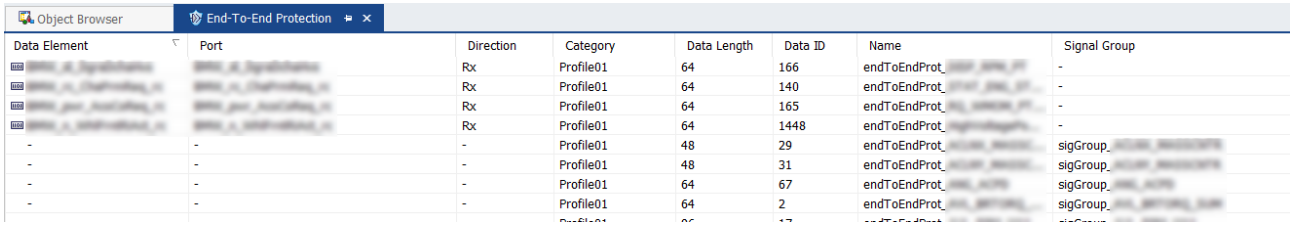
The DaVinci Developer Classic can show you a list of all the E2E protections as shown in below figure.



Data Element	Port	Direction	Category	Data Length	Data ID	Name	Signal Group
-	-	-	Profile01	64	2	endToEndProt_	sigGroup_
-	-	-	Profile01	64	9	endToEndProt_	sigGroup_
-	-	-	Profile01	56	11	endToEndProt_	sigGroup_
-	-	-	Profile01	40	12	endToEndProt_	sigGroup_
-	-	-	Profile01	64	13	endToEndProt_	sigGroup_
-	-	-	Profile01	48	14	endToEndProt_	-
-	-	-	Profile01	96	17	endToEndProt_	sigGroup_
-	-	-	Profile01	48	29	endToEndProt_	sigGroup_
-	-	-	Profile01	48	31	endToEndProt_	sigGroup_
-	-	-	Profile01	64	32	endToEndProt_	-
-	-	-	Profile01	64	53	endToEndProt_	sigGroup_
-	-	-	Profile01	64	55	endToEndProt_	sigGroup_
-	-	-	Profile01	40	57	endToEndProt_	sigGroup_
-	-	-	Profile01	72	58	endToEndProt_	sigGroup_
-	-	-	Profile01	32	63	endToEndProt_	sigGroup_

1.1.1 Figure 2-49 DaVinci Developer Classic View of all E2E Protections

The E2E protection is associated with a signal group. If you enable the E2E protection after establishing a data mapping, you will find a corresponding line in the same view with that information as shown in the next figure.



Data Element	Port	Direction	Category	Data Length	Data ID	Name	Signal Group
		Rx	Profile01	64	166	endToEndProt_	-
		Rx	Profile01	64	140	endToEndProt_	-
		Rx	Profile01	64	165	endToEndProt_	-
		Rx	Profile01	64	1448	endToEndProt_	-
-	-	-	Profile01	48	29	endToEndProt_	sigGroup_
-	-	-	Profile01	48	31	endToEndProt_	sigGroup_
-	-	-	Profile01	64	67	endToEndProt_	sigGroup_
-	-	-	Profile01	64	2	endToEndProt_	sigGroup_

Figure 2-50 DaVinci Developer Classic View of E2E Protections Associated with a Port

For each E2E protection that you map to a port, the DaVinci Developer Classic creates a copy of the E2E protection with the prefix Ext_. For example, if you enable the E2E protection **endToEndProt_<ExampleSignalGroupName>_LH**, you will see a second entry in this view with the name **Ext_endToEndProt_ <ExampleSignalGroupName>_LH**.

2.5.1.1 Creating a Data Mapping with an E2E Protection

For signal groups with an associated E2E protection, you will see the prompt shown in Figure 2-51, when you create the data mapping:

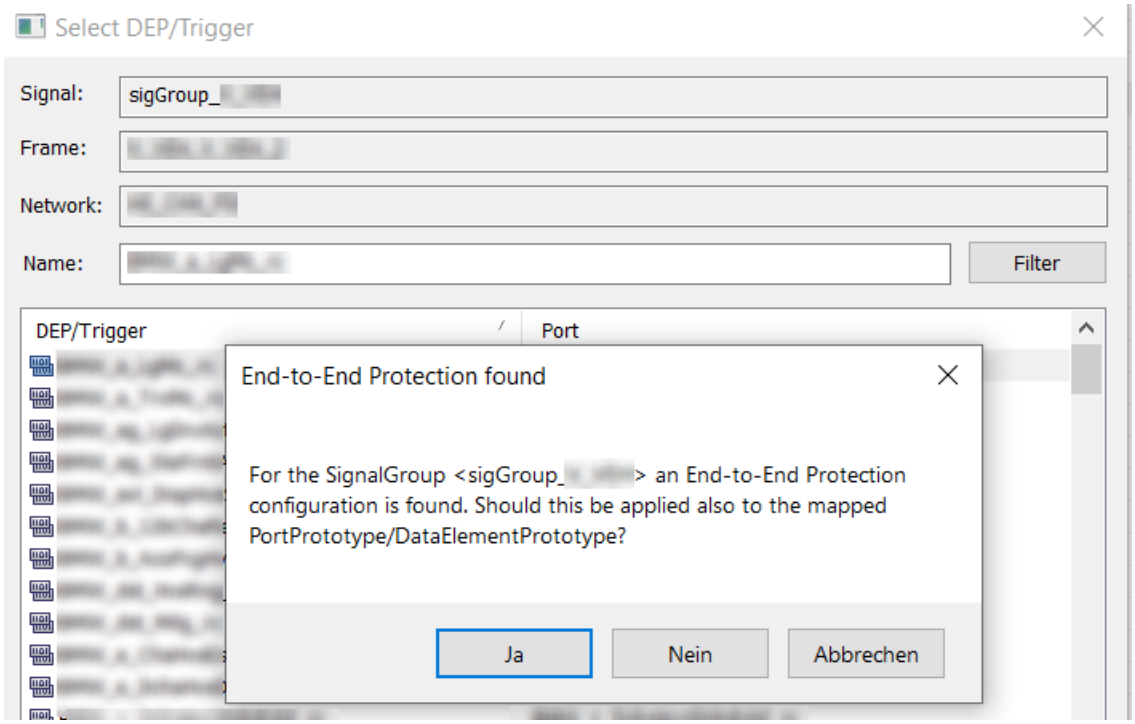


Figure 2-51 Prompt to Establish the E2E Protection for This Data Mapping

Caution

After you add E2E protection to a port you must synchronize the DaVinci Developer Classic workspace with your DaVinci Configurator Classic!

Open the DaVinci Configurator Classic and execute the solving action RTE59000.

The E2E protected data mappings are created in the same way as every other data mapping (cf. Figure 2-).

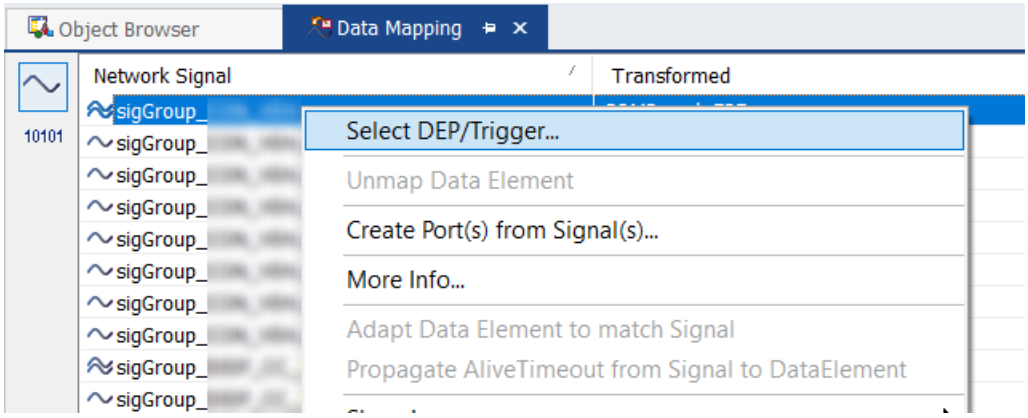


Figure 2-52 Creating a Data Mapping for a Signal Group

2.5.2 Step 2: Create Invariant System Extracts for All Four Variants

The E2EPW generator does not support multiple variants. Therefore, we must use the DaVinci Configurator Classic to export for system extracts that do not contain only the information of a single variant each. You can create these invariant system extracts via the DaVinci Configurator Classic command line. Open a Windows command prompt in the directory, where your .dpa file is located and execute the following command (on a single line):

```
…\CBD1500330_D02_MPC57xx\DaVinciConfigurator\Core\DVCfgCmd.exe
-p SRR_MultiConfig.dpa
--exportDir e2eout
--exportPostbuildVariants
--exportWithExporterId sysexForRteGenerator
```

Caution
You must create the output directory (parameter –exportDir) first! In the example invocation above, the output directory is called e2eout. .

After this command finished, you will see a directory e2eout the content shown in Figure 2-34

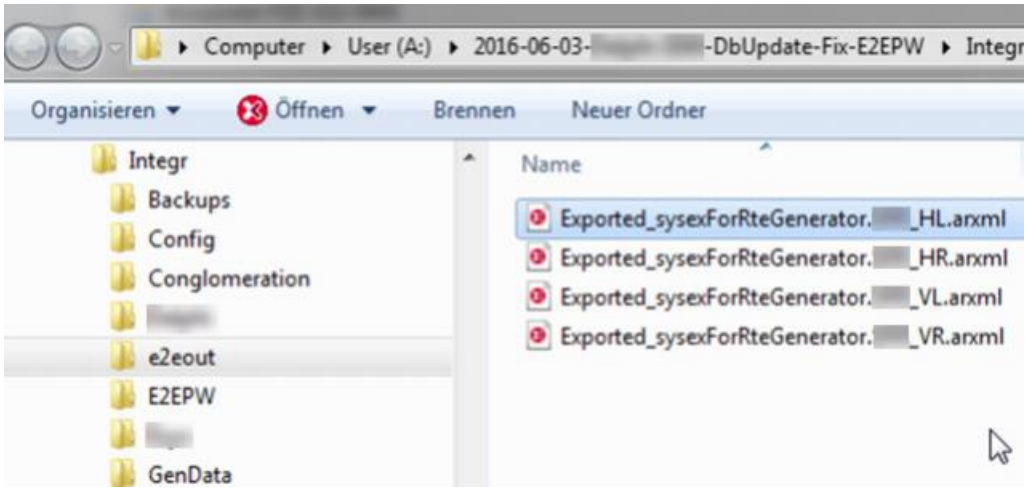


Figure 2-53 The invariant System Extracts Exported by the DaVinci Configurator Classic

2.5.2.1 Filtering of the Exported System Extracts

The system extracts that the DaVinci Configurator Classic exported contain all E2E protections. This confuses the E2EPW generator, because the data mappings for the E2E protections are variant and therefore missing in this invariant system extracts.

Before you can call the E2EPW generator, you must filter the exported system extracts so that each system extract includes only those E2E protections for that specific variant.

Now, there are no E2E protections configured for the variant rear-left. Therefore, you can delete the file Exported_sysexForRteGenerator.xxx_HL.arxml.

For all the other three files, you have to edit them.

Note
The editing of these files is best done with an XML editor that supports a grid view.

Search for the ARXML element with SHORT-NAME **E2EPROTECTIONSET** (cf. Figure 2-35)

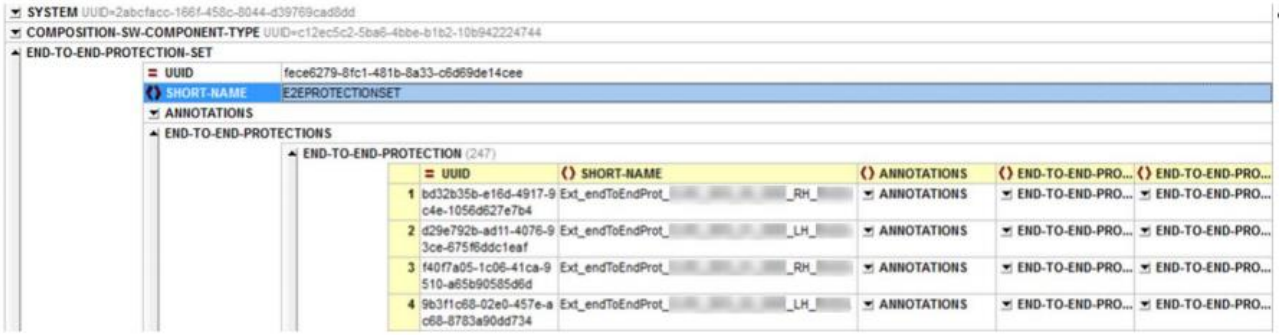


Figure 2-54 End-to-End Protections in the Exported System Extracts

Now delete all the END-TO-END-PROTECTION elements that do not belong to this variant. You can find all the E2E protections for each variant in your system architecture tool output or you can try to identify it by the name (e.g. HL is most likely relevant for the variant rearleft only).

© 2022 Vector Informatik GmbH

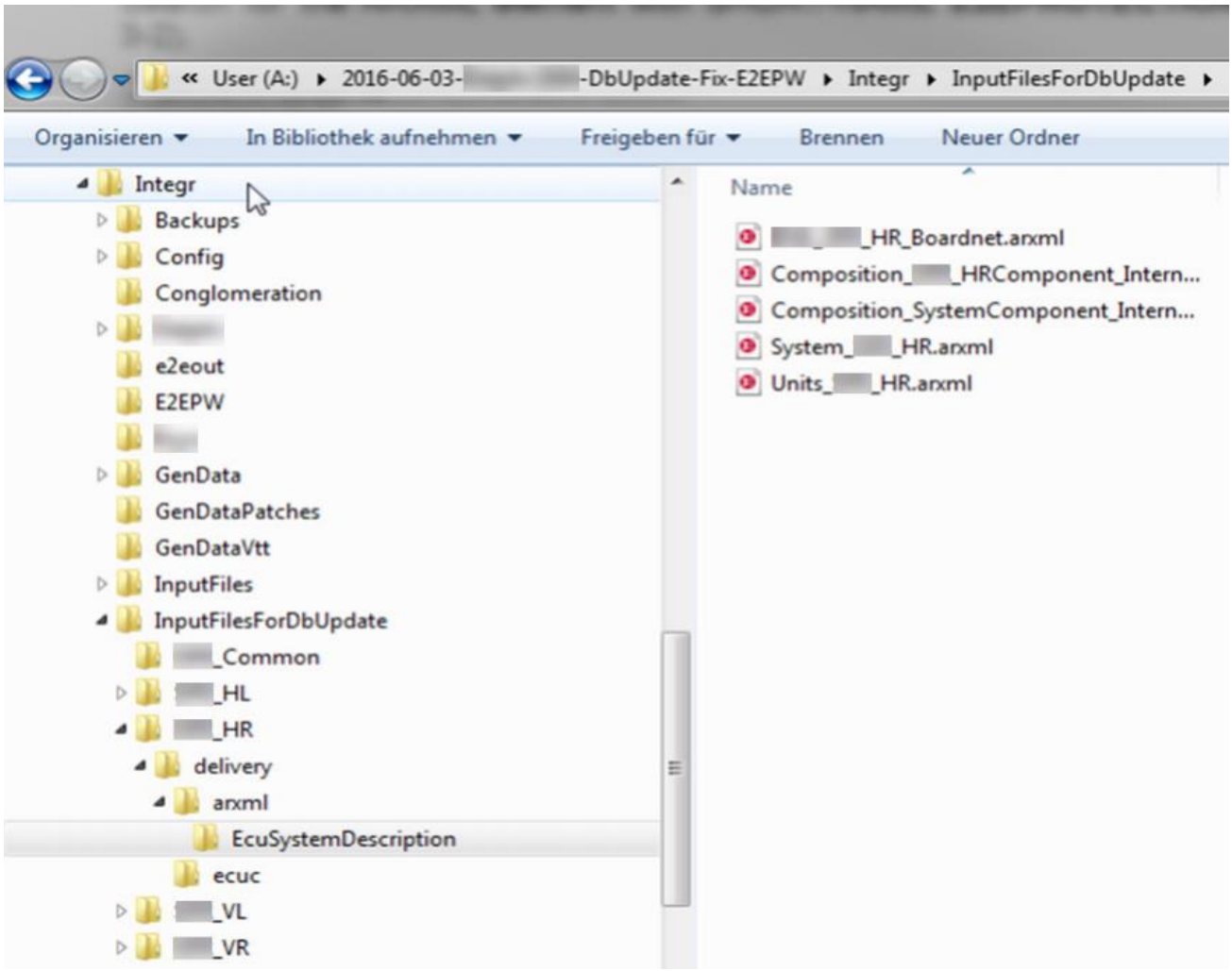


Figure 2-55 The Output for the Master Variant

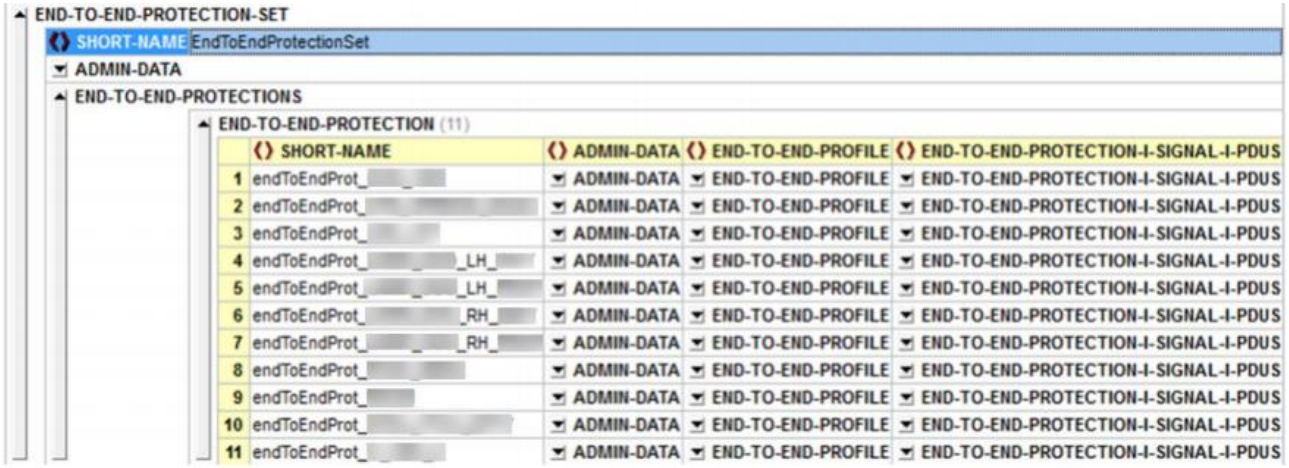


Figure 2-56 The E2E Protections for the Master (Shown in Grid View of Altova XmlSpy)

At the end of this step, you should have only the E2E protections remaining in the exported system extracts that belong to this variant.

2.5.3 Step 3: Create Configuration Files for the E2EPW Generator

From the exported system extracts of step 2, you can create the configuration files for the E2EPW generator. These configuration files include the necessary information to generate the E2E protection wrapper implementations. There is a dedicated tool for this step included in your BSW Package delivery from Vector:

```
…< SIP-Folder>\CBDxxxxxxx_D0z\Generators\E2EPW\pwg_preprocessor.exe
```

You must invoke this tool once for each of the three variants: xxx_HR, xxx_VR and xxx_VL.

xxx_HR

```
pwg_preprocessor.exe -f -v2 -b BIG_ENDIAN SRR_HR
e2eout\Exported_sysexForRteGenerator.xxx_HR.arxml e2eout
```

xxx_VR

```
pwg_preprocessor.exe -f -v2 -b BIG_ENDIAN SRR_VR
e2eout\Exported_sysexForRteGenerator.xxx_VR.arxml e2eout
```

xxx_VL

```
pwg_preprocessor.exe -f -v2 -b BIG_ENDIAN SRR_VL
e2eout\Exported_sysexForRteGenerator.xxx_VL.arxml e2eout
```

After these steps, there are three .cfg files in the e2eout directory (cf. Figure 2-38)

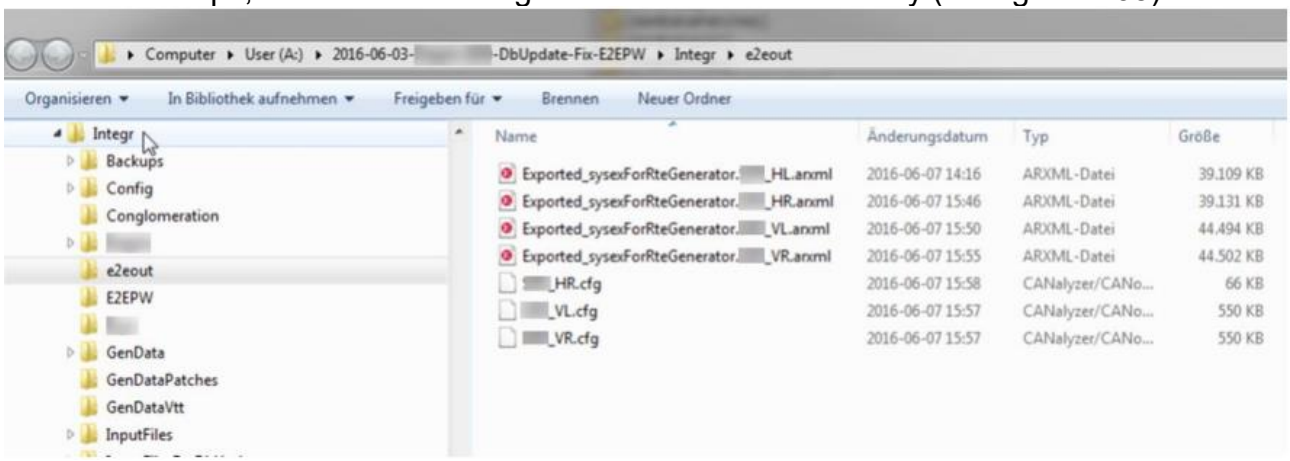


Figure 2-57 The Generated .cfg Files for the E2EPW Generator

2.5.4 Step 4: Generate the E2E Protection Wrappers

From the generated .cfg files of step 3, you can create the E2E protection wrappers using the E2EPW generator.

There is a dedicated tool for this step included in your BSW Package delivery from Vector:

```
…< SIP-Folder>\CBDxxxxxxx_D0z\Generators\E2EPW\pwg.exe
```

You must invoke this tool once for each of the three variants: xxx_HR, xxx_VR and xxx_VL.

Caution
you must create the output directory first. In the examples below, the output directory is called E2EPW.

SRR_HR
```
pwg.exe e2eout\xxx_HR.cfg E2EPW
```
SRR_VR
```
pwg.exe e2eout\xxx_VR.cfg E2EPW
```
SRR_VL
```
pwg.exe e2eout\xxx_VL.cfg E2EPW
```

After these steps, you find see the generated E2E protection wrappers in the E2EPW directory (cf. Figure 5-1).

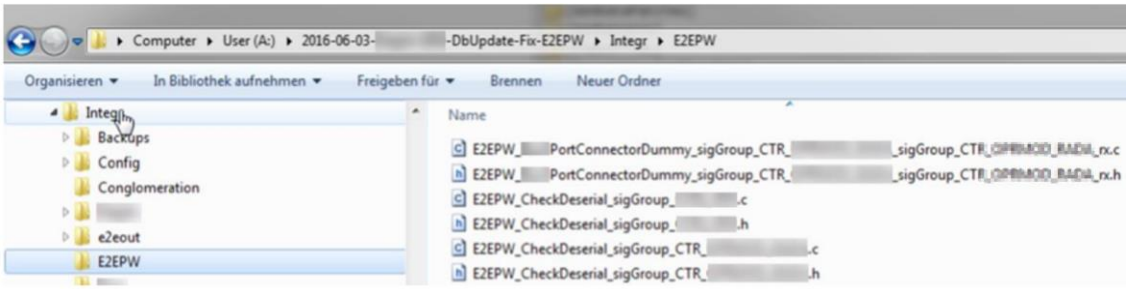


Figure 2-58 The Protection Wrappers Generated by the E2EPW Generator

2.6 Configuration of E2EPW Using an LDF File

This chapter explains how to generate End-to-End protection wrapper (E2EPW) solution from a LIN Description File (LDF). LDF is the typical description file for communication via a LIN bus.

The workflow of creating the E2e configuration can be separated into the following 3 steps:

1. Identifying Signals from a LIN Frame to be E2E protected
2. Optional: Converting LDF into AR SystemDescription using Vector Legacy Converter (VLC)
3. Configuration of E2E using DaVinci Developer Classic.

2.6.1 Step 1: Identifying Signals from a LIN Frame to be E2E Protected

There are multiple E2E Profiles available in AUTOSAR. Because of the LIN frame size, only Profile01 and Profile02 can be selected. All profiles require to have mandatory elements,

such as checksum and counters, inside of a SignalGroup. The following LIN frame should support to understand the parts of such profile:

Signal	StartBit
ChecksumMgnSens	0
CounterMgnSens	8
BrakeValue	12
SwitchValue	16
ChecksumLightFence	24
CounterLightFence	32
LightFenceFRStatus	36
LightFenceFLStatus	38
DiagData	40
ResponseError	47

Table 2-59 List of Signals in a LIN Frame Considered for E2E Profile

In the Table 2-59 above, each signal is identified by an offset counted from the start of the frame. As there are two sets of counters and checksums, there are two individually protection data groups inside of one frame: barrier

- SG_MgnSens
- SG_LightFence

In the following example it is assumed that the SignalGroup **MgnSens** is containing the signals **SwitchValue** and **BrakeValue**. Additionally, the elements of the SignalGroup **LightFence** are **LightFenceFRStatus** and **LightFenceFLStatus**. The grouping of the signals is shown inside of the following table:

SignalGroup	Signals	Data / E2E information (E2e)
SG_MgnSens	BrakeValue	Data
	SwitchValue	Data
	ChecksumMgnSens	E2e
	CounterMgnSens	E2e
SG_LightFence	LightFenceFRStatus	Data
	LightFenceFLStatus	Data
	ChecksumLightFence	E2e
	CounterLightFence	E2e

Table 2-60 List of Signals in a LIN Frame Considered for E2E Profile

In order to create SignalGroups for the above identified groups, during the Import of LDF file into DaVinci Configurator Classic, you need to create a VSDE file. This file provides additional rules to the Vector Legacy Converter, e.g. that a certain SignalGroup should be defined.

Following `LDF_SigGrp_New.vsde` file creates the SignalGroup **SG_MgnSens** inside of your configuration (in DaVinci Configurator Classic and DaVinci Developer Classic), if it is imported together with the LDF, which is holding the referenced signals.

LDF_SigGrp_New.vsde

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <EXTRACT-EXTENSION xsi:schemaLocation="http://www.vector-informatik.de/ExtractExtension ExtractExtension.xsd" xmlns
3
4    <SYSTEM-SIGNAL-GROUP>
5    <SHORT-NAME>SG_MgnSens</SHORT-NAME>
6    <SYSTEM-SIGNAL-REFS>
7      <SYSTEM-SIGNAL-REF>ChecksumMgnSens</SYSTEM-SIGNAL-REF>
8      <SYSTEM-SIGNAL-REF>CounterMgnSens</SYSTEM-SIGNAL-REF>
9      <SYSTEM-SIGNAL-REF>SwitchValue</SYSTEM-SIGNAL-REF>
10     <SYSTEM-SIGNAL-REF>BrakeValue</SYSTEM-SIGNAL-REF>
11   </SYSTEM-SIGNAL-REFS>
12 </SYSTEM-SIGNAL-GROUP>
13 </EXTRACT-EXTENSION>
```

Figure 2-61 Content of VSDE Extension file for Signals Grouping

2.6.2 (Optional) Step 2: Converting LDF Into AUTOSAR SystemDescription ARXML

For debugging purpose or if you would like to import the SystemDescription to DaVinci Developer Classic directly, you might want to create a system description out of the LDF (and VSDE). By using the Vector Legacy Converter (VLC), it is possible to transform an LDF into an AUTOSAR System Description ARXML. The command line tool can usually be found within the following path:

```
..<SIP-Folder>\Generators\LegacyDb2SystemDescrConverter\
```

Invoking the following command within the command line converts the LDF (and VSDE) into a SystemDescription file:

```
LegacyDb2SystemDescrConverter -e -r 40 -o
.\Output\Extract_Output_vsde.arxml .\Input\LIN_19200_2.2_V0.10.ldf
.\Input\LDF_SigGrp_New.vsde
```

How to use the command line tool:

```
LegacyDb2SystemDescrConverter [options] <file|dir> [<file|dir>
...] [extfile]
```

The options and parameters are described as follows:

```
-e Create ECU extract

-r AUTOSAR Release (available version depends on BSW Package, use -
h to identify possible options)

-o Output file or directory

<file> Input file (*.dbc, *.ldf, *.xml) or directory

[extfile] Extension file (*.vsde)
```

The above-mentioned command creates `Extract_Output_vsde.arxml` along with other ARXMLs, whose ECU nodes are provided in the input LDF `Legacy_Input.ldf`. The selected AUTOSAR scheme is selected to be 4.0.3 in this specific case. You may

refer to **Fehler! Verweisquelle konnte nicht gefunden werden.** for more details about t he options and parameters.

In the following screenshot Figure 2-34 the successful execution of the above-mentioned command is shown. In order to execute the command line tool no BSW Package license is required, so the yellow marked warning can be ignored.

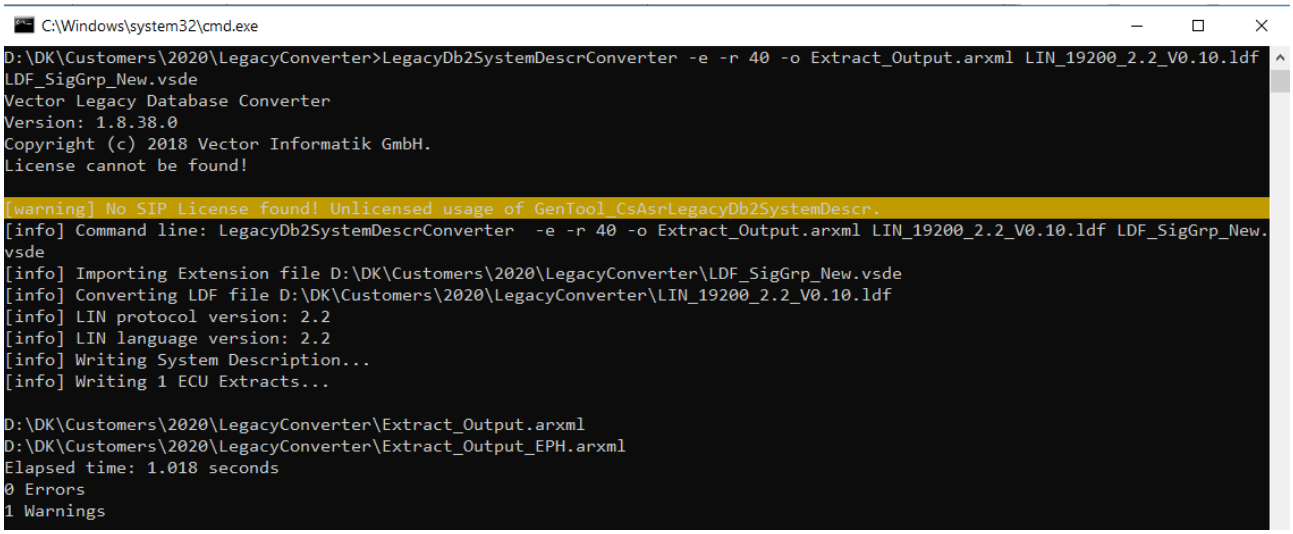


Figure 2-34 Invoking Vector Legacy Converter from Command Line Window with VSDE File as one of the Parameters

For further debugging you might be interested in the comparison between the ARXML with and without SignalGroup information. For generating both files, you can execute the command line tool once with - and once without - VSDE file argument.

The following screenshot shows the comparison between the two ARXML files. On the left you can see the SystemDescription without SignalGroup information (from vsde file), on the right side the SignalGroup is derived successfully.

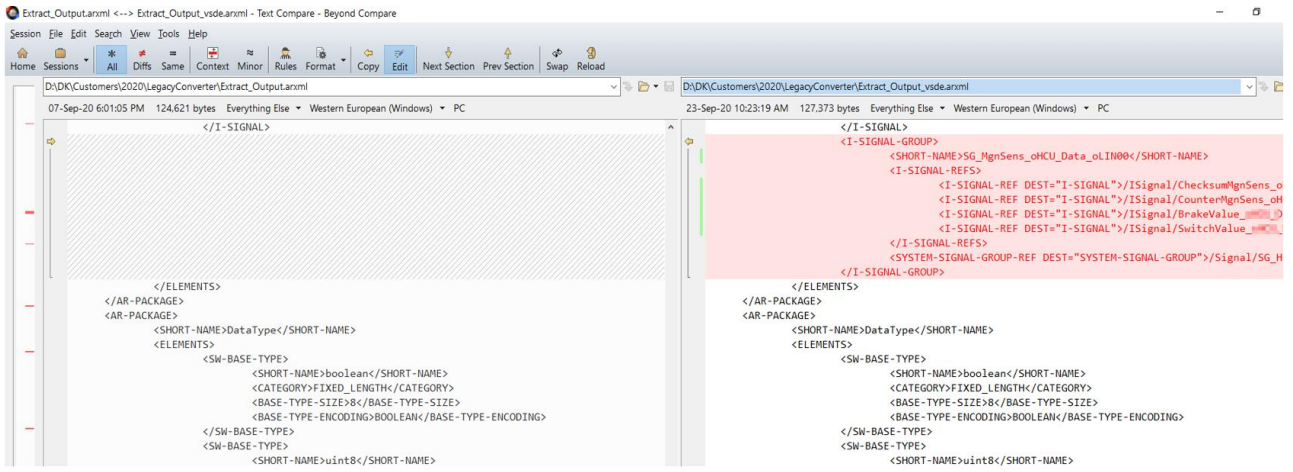


Figure 2-35 Comparison of ECU Extracts with VSDE Extension File as one of Parameters to Legacy Converter



Expert Knowledge
It is possible to modify the ECU Extract manually using AUTOSAR XML Editor to create SignalGroups.

Please be aware that modifying XML files manually is quite error prone. Therefore, applying the VSDE file and creating such information automatically should be preferred.

2.6.3 Step 3: Configuration of E2E Using DaVinci Developer Classic

The LDF with VSDE file can be used as an import to the DaVinci Configurator Classic project. This process is described in chapter 2.1.4 and valid if your workspace contains a DaVinci Configurator Classic project and DaVinci Developer Classic workspace.

Otherwise, the SystemDescription, which was generated in the previous steps, can be imported into DaVinci Developer Classic as shown in Figure 2-3 and Figure 2-4. After the description was successfully imported and you performed the signal mapping, the E2E protection needs to be configured.

The process of configuring E2ePw in the DaVinci Developer Classic is described in chapter 2.1.3, which you can reference for further information about the actual E2e configuration.

2.7 AUTOSAR Specification of E2E Protection (AUTOSAR Classic 4.2.2)

In the AUTOSAR System Template, version 4.2.2 (AUTOSAR, 2015), you can find an overview about the elements necessary to describe an EndToEndProtectionSet properly. In the following graphic, this will be described in detail

(Source AUTOSAR specification (AUTOSAR, 2015))

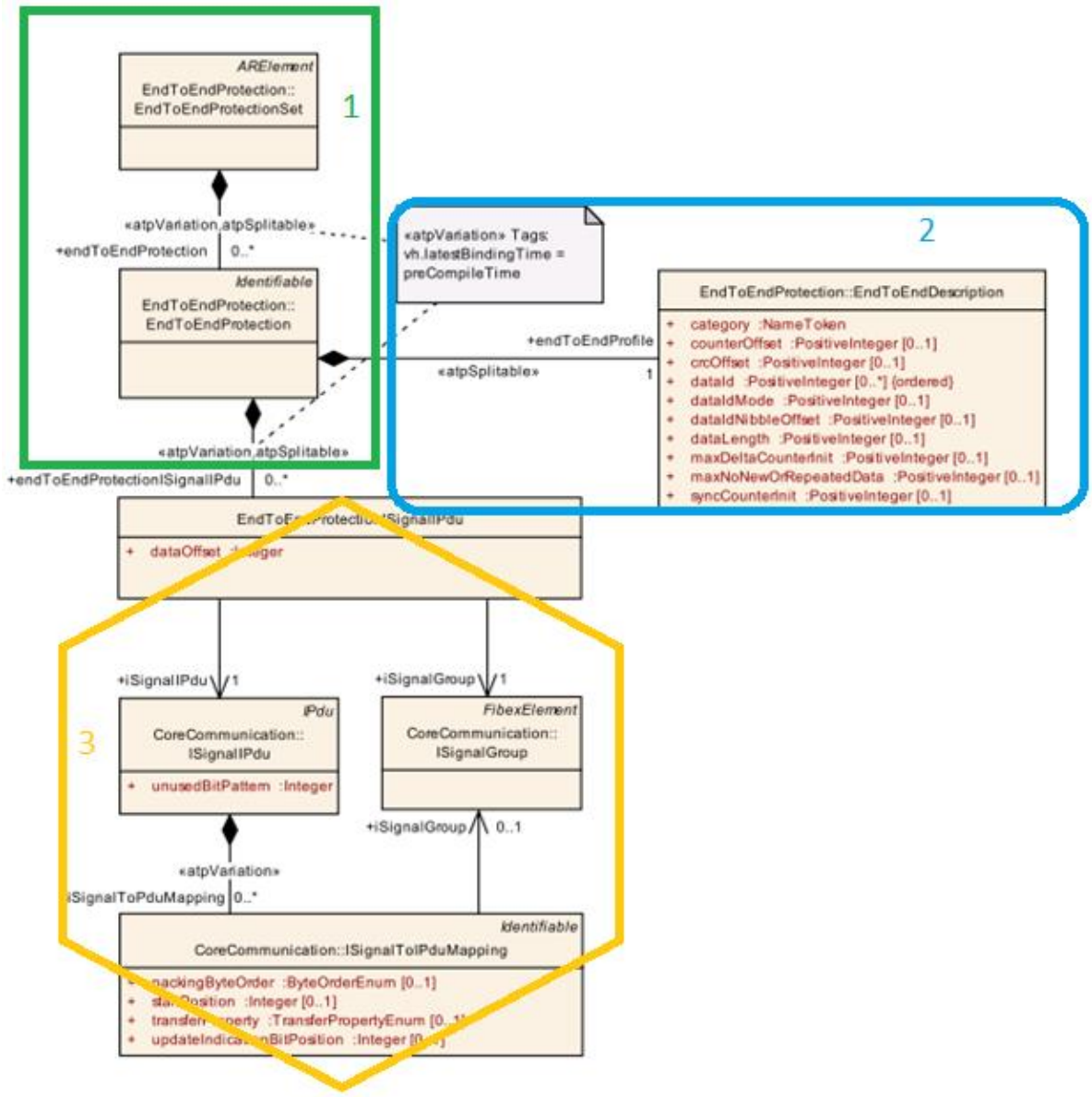


Figure 2-36 EndtoEndProtection for COM IPdus

1. The AR-Element **EndToEndProtectionSet** can hold an arbitrary number of (or none) **EndToEndProtection** identifiables. The Identifiable **EndToEndProtection** is a composition out of exactly one **EndToEndDescription** (Profile) and an arbitrary number of **EndToEndProtectionISignalIPdus**.
2. The **EndToEndProtectionDescription**, also called E2E Profile, defines the setup of the End-To-End Protection. Beside the signals, that the protection is configured for, the profile contains all parameter. Next to the **Category** of type **NameToken** all parameter

is optional and may be defined at most once. Some parameter become mandatory depending on the profile that is used. This specific is not further discussed in the following.

3. The last part, the **EndToEndProtectionSignalIPdu**, describes the mapping of the Signalgroup to an iSignalPdu, which is used for sending. Tor this purpose exactly one iSignalGroup and also exactly one iSignalIPdu gets referenced. Inside of the Pdu next to the data a CRC (checksum) and a dataOffset (counter) is add – please consider reserving the according bits to be available (unusedBitPattern).



Be aware about the following limitation:
[UC_E2E_00255] ⌈ If the E2E Library is invoked at the level of data elements and 1:N communication model is used and the data elements are sent using more than one I-PDU, then all these I-PDUs shall have the same layout. ⌋ (SRS_E2E_08528).

2.8 Practical Example Using a Manually Created Configuration

This chapter gives a practical example, using a manually created AUTOSAR description, for the earlier described AUTOSAR description.

The E2E Protection is inside of an own AR-Package. This package has the short-name **EndToEndProtections** in this example. Inside there is exactly one protection set **EndToEndConnections**, which structure is discussed in the following chapter.

1. The AR-Element **EndToEndProtectionSet** („EndToEndConnections“) containes an arbitrary number of **EndToEndProtection** identifiables. The **EndToEndConnection** is built out of a name, the profile, and the variable prototypes that should be protected. In

the following example, the first identifiable with the short-name **ECU_Composition_...** is explained in detail.



Figure 2-37 AR-Package EndToEndProtections

2. The **EndToEndProtectionDescription**, also called E2E Profile, is used to hold the configuration of an End-To-End protection. In this specific example, the Profile 2 is used, which is represented by the value **PROFILE_2** of parameter **Category**.



Figure 2-38 Configuration for an EndToEndProtectionDescription

3. The final part, the **EndToEndProtectionSignalIPdu**, shows the mapping between **Signalgroup** and **iSignalIPdu** that sends the according Signalgroup to the bus system. Therefore, a variable protoype is referenced. In the specific example, there is exactly one **END-TO-END-PROTECTION-VARIABLE-PROTOTYPE** referenced inside

of **END-TO-END-PROTECTION-VARIABLE-PROTOTYPES** as you can see in the screenshot below.

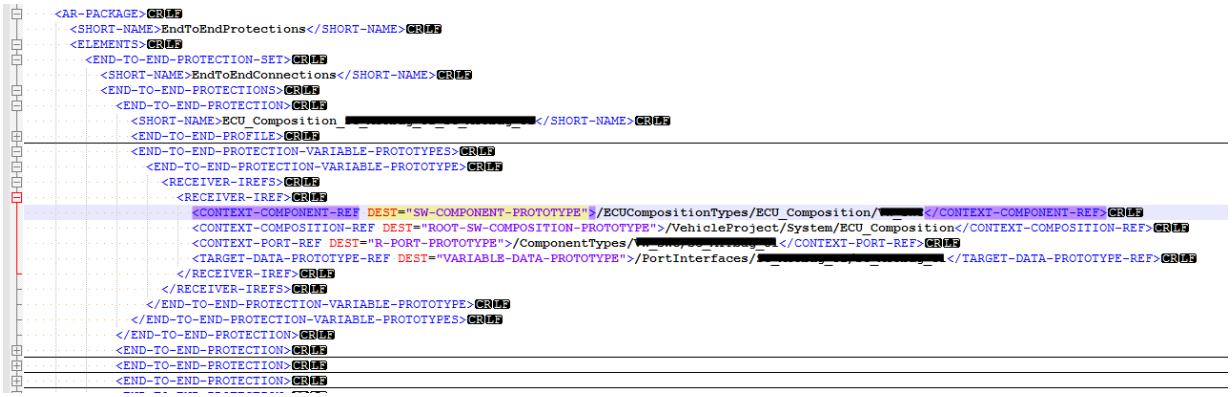


Figure 2-39 Configuration for EndToEndProtectionSignalIPdu

3 E2E Transformer Integration

3.1 Configuration using Different Input files

E2E Transformer can be integrated with different types of Input files such as legacy DBC file and System extract. Under this section you will discover how to configure E2E Transformer using different input files.

Caution
It is not possible to use E2eXf with other input files than ARXML or DBC file (e.g. LDF, Fibex,..). Please advise your customer or process partner in this case.

3.1.1 E2EXf configuration using ECU extract (preferred)

To use the E2E Transformation mechanism, AUTOSAR specifies that the E2E-relevant information should be **first** provided inside the System Extract. See below a screenshot of a SystemDescriptionExtract.arxml with one of the needed information provided for an ISignal or ISignalGroup:



Figure 3-1 AR Configuration for an E2E Transformer ISignal or ISignalGroup

Expert Knowledge
It is very recommended to use PREEvision to add all E2E-relevant information inside an extract. A manual editor is not recommended.

3.1.2 Configuration of E2eXf Using a DBC File

In order to use DBC file for configuration of E2EXf there must be some mandatory Attributes defined in database file (see Table 3-1). Based on these attributes the required AR-Package structures are added to the SystemDescriptionExtract. This is performed silently during the import into your configuration in DaVinci Configurator Classic (silently via the command line tool Vector Legacy Converter, which transforms the DBC into ARXML).

First, please verify that all required attributes are already defined by using Vector CANdb++ Editor. Navigate to **View | Attribute Definitions** to get a list of available attributes of your DBC file. Inside the below screenshot you find an example, which shows all attributes necessary for configuration of E2E Profile 05. If you require another profile, please adapt the values accordingly.

You can either import the attributes from another DBC file or add new Attributes manually by right click and select New to provide a fully new definition of an Attribute.

Vector CANdb++ Editor - D:\usr\usage\Delivery\CBD18x\CBD1800943\D03_ALM\external\Demo\Database\CAN_0.dbc - [Attribute Definitions]

File Edit View Options Window Help

Type Of Object	Name	Value Type	Mini...	Max...	Default
Network	Baudrate	Integer	0	100...	500000
Network	BusType	String	-	-	
Message	DiagConnection	Integer	0	655...	0
Message	DiagRequest	Enumeration	-	-	No
Message	DiagResponse	Enumeration	-	-	No
Message	DiagState	Enumeration	-	-	No
Message	DiagUUDTResponse	Enumeration	-	-	No
Message	E2EDataId	Integer	0	0	0
Message	E2EDataLength	Integer	0	0	0
Network	E2EP05MaxDeltaCounter	Integer	0	0	2
Network	E2EP05MaxErrorStateInit	Integer	0	0	2
Network	E2EP05MaxErrorStateInvalid	Integer	0	0	2
Network	E2EP05MaxErrorStateValid	Integer	0	0	2
Network	E2EP05MinOkStateInit	Integer	0	0	2
Network	E2EP05MinOkStateInvalid	Integer	0	0	2
Network	E2EP05MinOkStateValid	Integer	0	0	2
Network	E2EP05Offset	Integer	0	0	0
Network	E2EP05ProfileName	Integer	0	0	0
Network	E2EP05UpperHeaderBitsToShift	Integer	0	0	0
Network	E2EP05WindowSize	Integer	0	0	4
Message	E2EProfile	String	-	-	
Message	GenMsgCycleTime	Integer	0	0	0
Message	GenMsgCycleTimeFast	Integer	0	0	0

Figure 3-2 DB Configuration for the Attributes for E2E Profile 05

The following attributes have <u>Message</u> or <u>Signal</u> as "Type of Object"(depending on your project)

- E2EDataId (Integer)
- E2EDataLength (String)
- E2EProfile (String)

The following attributes have Network as "Type of Object"
Note: not all attributes may be required for each profile.

- E2EP0XMaxDeltaCounter (Integer)
- E2EP0XMaxErrorStateInit (Integer)
- E2EP0XMaxErrorStateValid (Integer)
- E2EP0XMaxErrorStateInValid (Integer)
- E2EP0XOffset (Integer)
- E2EP0XProfileName (Integer)
- E2EP0XUpperHeaderBitsToShift (Integer)
- E2EP0XWindowSize (Integer)

Naming:

1. All Network Type attributes require the standardized Prefix "E2EP0X", X being the number of the profile. Example: "E2EP02MaxDeltaCounter", for Profile 2.
2. E2EProfile uses a standardized string, by the name of "PROFILE_XX". Example: "PROFILE_02", for Profile 2, or "PROFILE_05" for Profile 5.

After finishing the editing of your attribute definitions, you need to save the DBC file. Finally, you can select the message, which the E2E protection should be active for.

To do so, double click the message, and select the **Attributes** tab in the appearing window. As depicted in the Figure 3-3 below, you need to set the attribute values depending to the E2eProfile and specification, which you require for the corresponding E2E message.

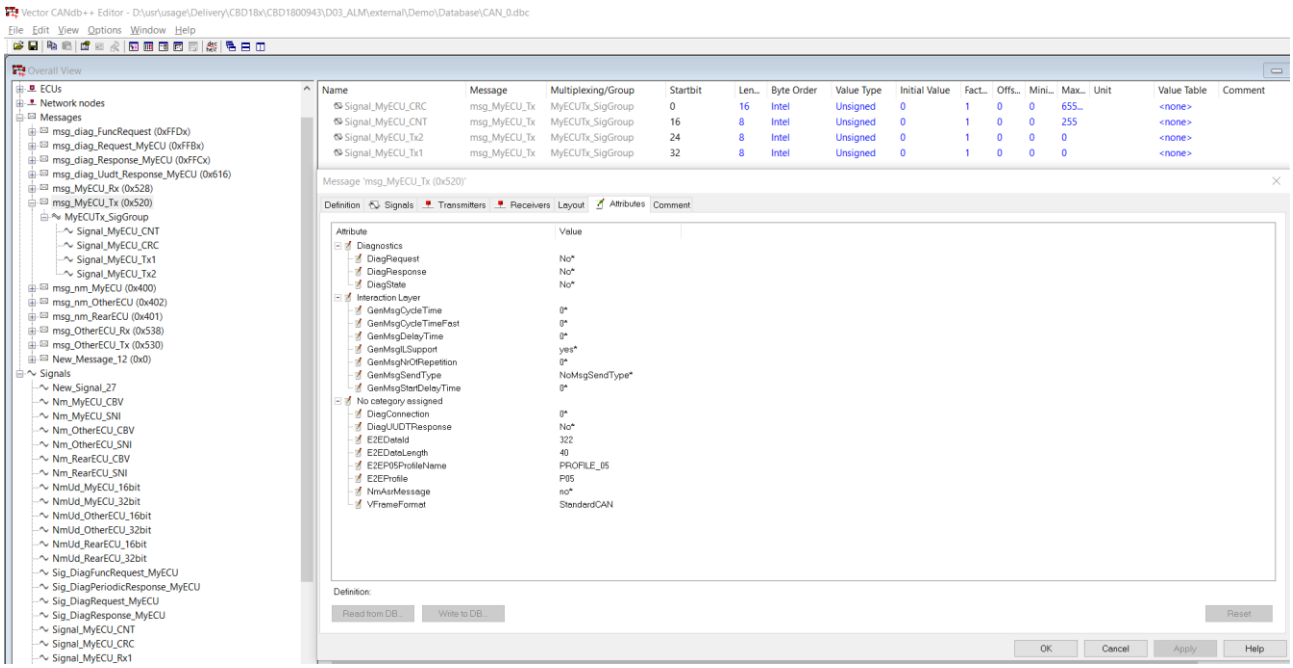


Figure 3-3 Verifying\Modifying the Attributes for Configuration of E2E Message

In Table 3-1 the DBC attributes, which are available for E2E configuration are described more detailed. There is a dependency between the required attributes and the profile you would like to use (not all attributes are required for each profile).

DBC Attributes	Description for E2E attributes
E2EProfile	Profile Short name for profile attribute.
E2EDataId	This represents a unique numerical identifier. Note: ID is used for protection against masquerading. The details concerning the maximum number of values (this information is specific for each E2E profile) applicable for this attribute are controlled by a semantic constraint that depends on the category of the EndToEndProtection. dataId is used as a unique identifier of a configuration object. One dataId can appear only once in the configuration.
E2EDataLength	For fixed size data: length of data in bits.
E2EP0XMaxDeltaCounter	Maximum allowed difference between the counter value of the current message and the previous valid message.
E2EP0XMaxErrorStateInit	Maximum number of checks in which ProfileStatus equal to E2E_P_ERROR was determined, within the last WindowSize checks, for the state E2E_SM_INIT.
E2EP0XMaxErrorStateValid	Maximum number of checks in which ProfileStatus equal to E2E_P_ERROR was determined, within the last WindowSize checks, for the state E2E_SM_VALID.
E2EP0XMaxErrorStateInValid	Maximum number of checks in which ProfileStatus equal to E2E_P_ERROR was determined, within the last WindowSize checks, for the state E2E_SM_INVALID.

E2EP0XOffset	Offset of the E2E header in the Data[] array in bits.
E2EP0XProfileName	This represents the identification of the concrete E2E profile. Possible profiles: 1 (only CP), 2 (only CP), 4, 5, 6, 7, 11, 22.
E2EP0XUpperHeaderBitsToShift	This attribute depends on the used serializing transformer: COM based transformer: 0 (no bits are shifted) SOME/IP transformer: 64 (to support the header shift of SOME/IP)
E2EP0XWindowSize	Size of the monitoring window (ProfileStatus circular buffer) for the state machine

Table 3-1 DBC Attributes Description in Accordance with System Description Extract for Profiles

After finishing the DBC modification, you can import the DBC file in Davinci Configurator Classic. Once you are done with Update workflow, you can verify the configured attribute values as shown below in Figure 3-4.

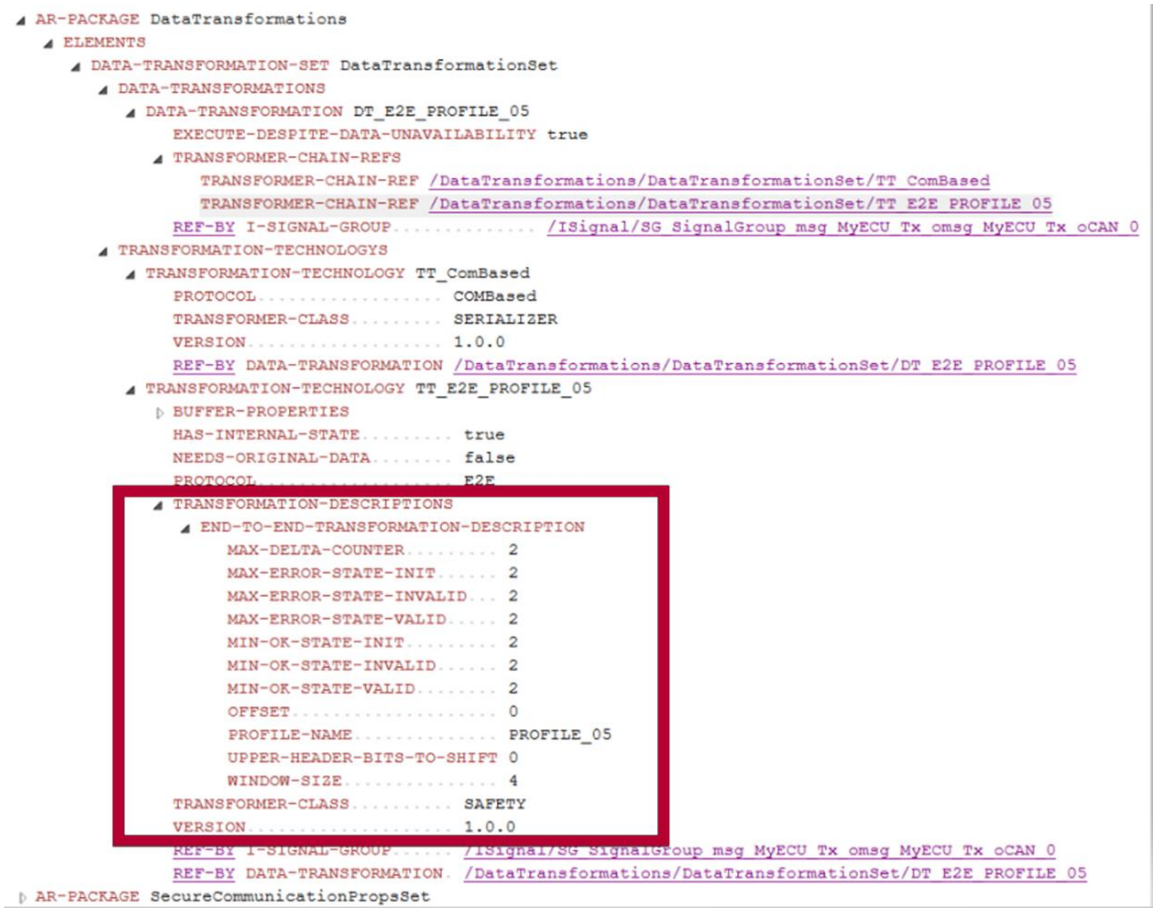


Figure 3-4 Imported Attributes in System Extract File.

Caution
It is not recommended to modify DBC file using a normal text editor, because there is a high risk of corrupting the DBC file. It is recommended to use Vector CANdb++ Editor.

3.1.3 Resulting System Description and Required configuration

The code extract below is an example of a SignalGroup with the relevant transformation information/labels. Have a look at the extract inside the test configuration (.\Config\System\SystemDescriptionExtract.arxml between lines 1381416 and 1381478). There you can find the keywords referring to ComXf and E2EXF.

```
1381415                 <SHORT-NAME>COP</SHORT-NAME>
1381416                 <ELEMENTS>
1381417                   <I-SIGNAL-GROUP UUID="BF3B0EAA1E2C374B92C4C90C5B84EEA5">
1381418                     <SHORT-NAME>[redacted]</SHORT-NAME>
1381419                     <LONG-NAME>
1381420                       <L-4 L="FOR-ALL">[redacted]</L-4>
1381421                     </LONG-NAME>
1381422                     <DESC>
1381423                       <L-2 L="DE">[redacted] </L-2>
1381424                       <L-2 L="EN">[redacted]</L-2>
1381425                     </DESC>
1381426                     <ADMIN-DATA>
1381427                       <DOC-REVISIONS>
1381428                         <DOC-REVISION>
1381429                           <REVISION-LABEL>0.0.0</REVISION-LABEL>
1381430                           <STATE>RELEASED</STATE>
1381431                           <DATE>2017-03-03T14:09:02Z</DATE>
1381432                         </DOC-REVISION>
1381433                       </DOC-REVISIONS>
1381434                       <SDGS>
1381435                         <SDG>
1381436                           <SDG-CAPTION>
1381437                             <SHORT-NAME>uuid</SHORT-NAME>
1381438                             <DESC>
1381439                               <L-2 L="EN">debug</L-2>
1381440                             </DESC>
1381441                           </SDG-CAPTION>
1381442                           <SD>SignalGroup: 3D9A29F257473D39AFDBDCA5EF99B4EB</SD>
1381443                         </SDG>
1381444                       </SDGS>
1381445                     </ADMIN-DATA>
1381446                     <COM-BASED-SIGNAL-GROUP-TRANSFORMATIONS>
1381447                       <DATA-TRANSFORMATION-REF-CONDITIONAL>
1381448                         <DATA-TRANSFORMATION-REF DEST="DATA-TRANSFORMATION">/DataTransformations/[redacted]/BaseTrans_Profile_5</DATA-TRANSFORMATION-REF>
1381449                       </DATA-TRANSFORMATION-REF-CONDITIONAL>
1381450                     </COM-BASED-SIGNAL-GROUP-TRANSFORMATIONS>
1381451                     <I-SIGNAL-REFS>
1381452                       <I-SIGNAL-REF DEST="I-SIGNAL">/ISignals/[redacted]/I-SIGNAL-REF>
1381453                       <I-SIGNAL-REF DEST="I-SIGNAL">/ISignals/[redacted]</I-SIGNAL-REF>
1381454                       <I-SIGNAL-REF DEST="I-SIGNAL">/ISignals/[redacted]/I-SIGNAL-REF>
1381455                       <I-SIGNAL-REF DEST="I-SIGNAL">/ISignals/[redacted]</I-SIGNAL-REF>
1381456                       <I-SIGNAL-REF DEST="I-SIGNAL">/ISignals/[redacted]</I-SIGNAL-REF>
1381457                       <I-SIGNAL-REF DEST="I-SIGNAL">/ISignals/[redacted]</I-SIGNAL-REF>
1381458                       <I-SIGNAL-REF DEST="I-SIGNAL">/ISignals/[redacted]</I-SIGNAL-REF>
1381459                       <I-SIGNAL-REF DEST="I-SIGNAL">/ISignals/[redacted]</I-SIGNAL-REF>
1381460                       <I-SIGNAL-REF DEST="I-SIGNAL">/ISignals/[redacted]</I-SIGNAL-REF>
1381461                       <I-SIGNAL-REF DEST="I-SIGNAL">/ISignals/[redacted]/I-SIGNAL-REF>
1381462                     </I-SIGNAL-REFS>
1381463                     <SYSTEM-SIGNAL-GROUP-REF DEST="SYSTEM-SIGNAL-GROUP">/SystemSignalGroups/[redacted]</SYSTEM-SIGNAL-GROUP-REF>
1381464                     <TRANSFORMATION-I-SIGNAL-PROPSS>
1381465                       <END-TO-END-TRANSFORMATION-I-SIGNAL-PROPS>
1381466                         <END-TO-END-TRANSFORMATION-I-SIGNAL-PROPS-VARIANTS>
1381467                           <END-TO-END-TRANSFORMATION-I-SIGNAL-PROPS-CONDITIONAL>
1381468                             <TRANSFORMER-REF DEST="TRANSFORMATION-TECHNOLOGY">/DataTransformations/[redacted]/E2E_Profile_5</TRANSFORMER-REF>
1381469                             <DATA-IDS>
1381470                               <DATA-ID>40</DATA-ID>
1381471                             </DATA-IDS>
1381472                             <DATA-LENGTH>40</DATA-LENGTH>
1381473                           </END-TO-END-TRANSFORMATION-I-SIGNAL-PROPS-CONDITIONAL>
1381474                         </END-TO-END-TRANSFORMATION-I-SIGNAL-PROPS-VARIANTS>
1381475                       </END-TO-END-TRANSFORMATION-I-SIGNAL-PROPS>
1381476                     </TRANSFORMATION-I-SIGNAL-PROPSS>
1381477                   </I-SIGNAL-GROUP>
1381478                 </ELEMENTS>
```

Figure 3-5 AR Configuration for SignalGroup with E2E Configuration

The E2EXf should be used in combination with the Serializer (ComXf or SomeIpXf). In DaVinci Configurator Classic these modules must be added but no configuration is required. The rest happens in DaVinci Developer Classic.

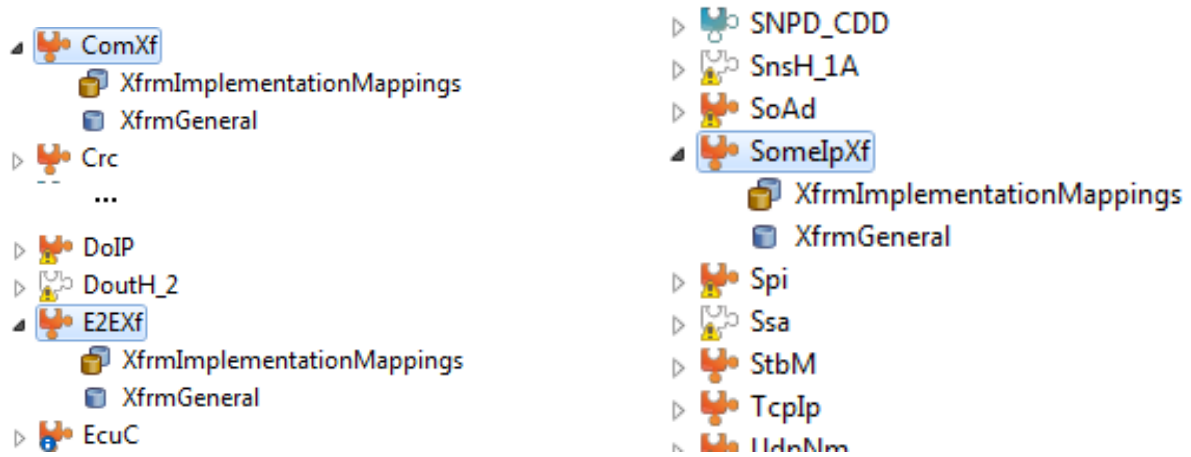


Figure 3-6 E2EXf Module in DaVinci Configurator Classic and SomeIpXf Module in DaVinci Configurator Classic

Some general configurations are needed in DaVinci Developer Classic. The first one concerns the workspace itself. Activate **Enable Data Transformation** first as follow:

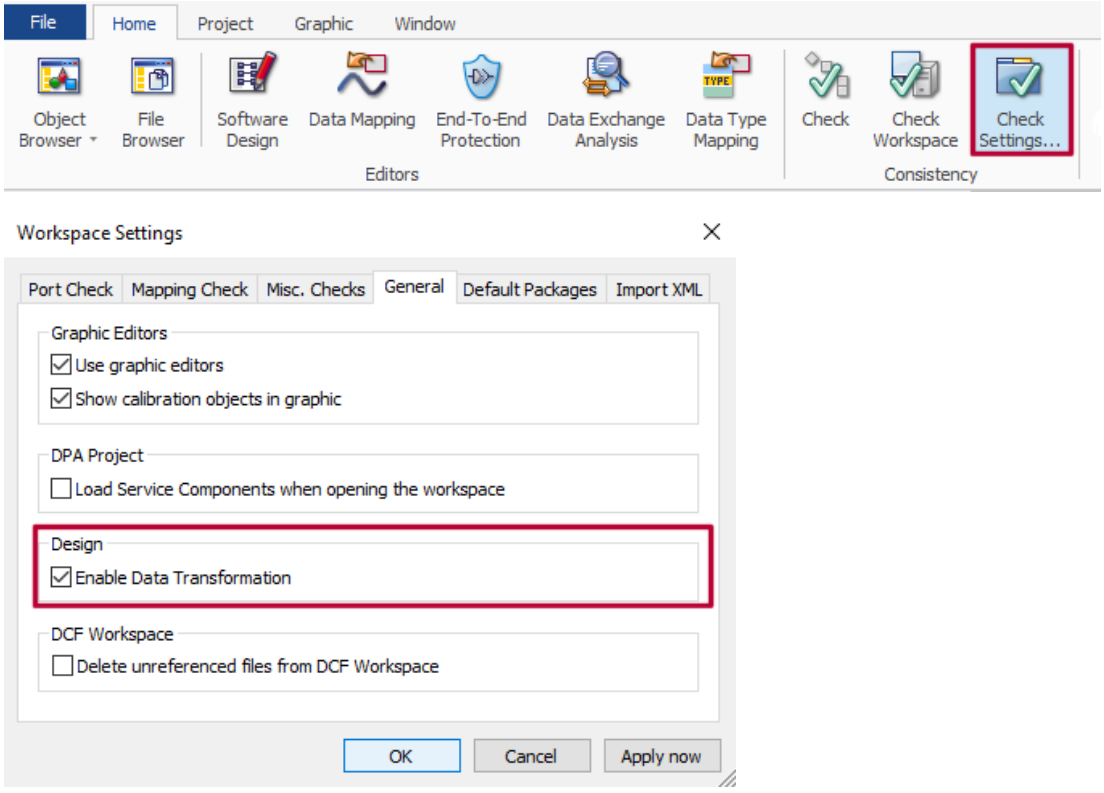


Figure 3-7 Enable Data Transformation in DaVinci Developer Classic Workspace

Caution
Do **NOT** activate **Use E2EXF**.

Now the column **Transformed** is displayed in configuration view of the **Data Mapping** (signal view). The column shows the type of transformation available for your signal or signal group.

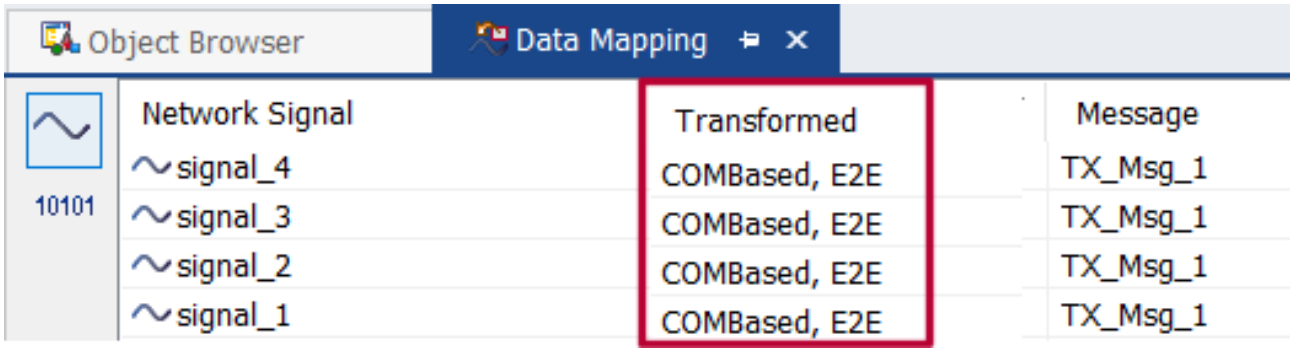


Figure 3-8 Network Signal Mapping in DaVinci Developer Classic Workspace

The E2E transformation can only be activated if a data mapping is performed on a dedicated Port Prototype (like for the E2EPW).

You can find more information in the **Properties** of the **Port Prototype**. You can decide to modify some additional transformation parameters. Normally. this information should be given by the customer. Anyway, any modification is possible if you select **End-to-End** as **Type** instead of **None**.

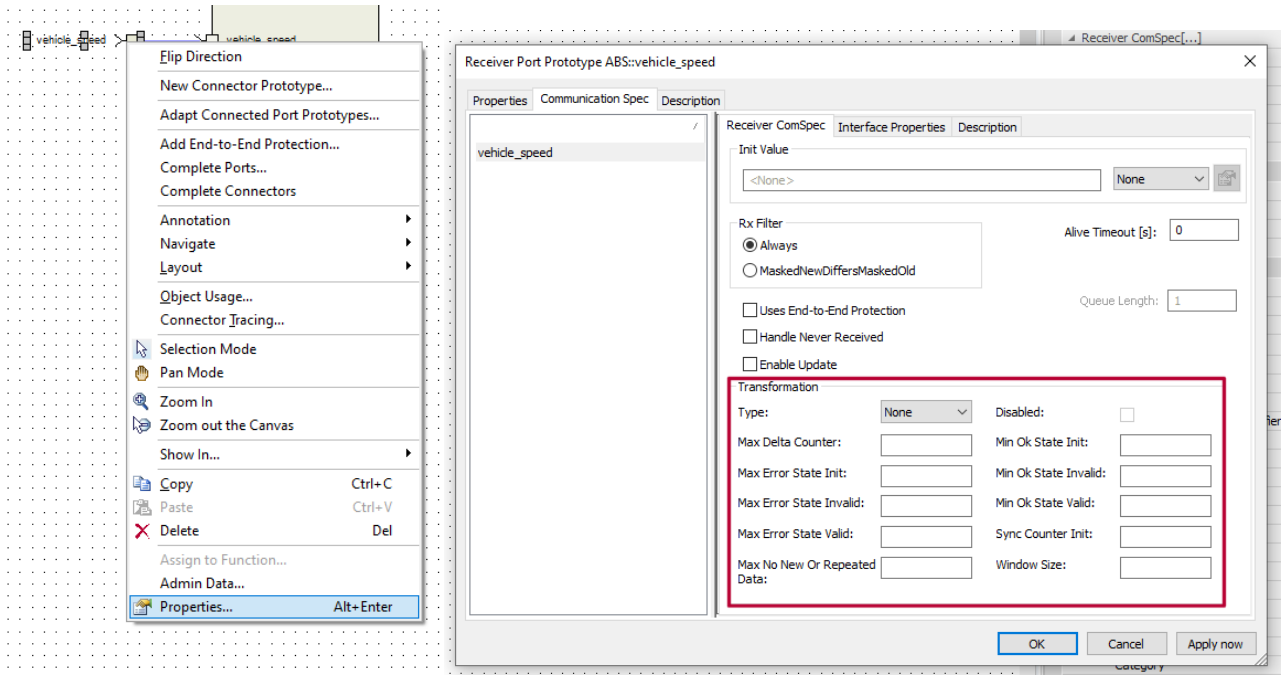


Figure 3-9 Configuring the Transformation Parameters

Meaning of Disables
All settings can be derived by its name except for the checkbox **Disabled**. With the checkbox **Disabled** it is possible to deactivate the E2E secure communication for a specific port interface to prevent the E2E and CRC mechanism from happening. To do so, set transformation **Type** to **End-to-End** and select **Disabled**. The return value will be `RTE_E_OK`.

No other configuration is needed.

You do **not** need to activate **Use End-To-End Protection** on a Port Prototype – this is for the E2EPW only.

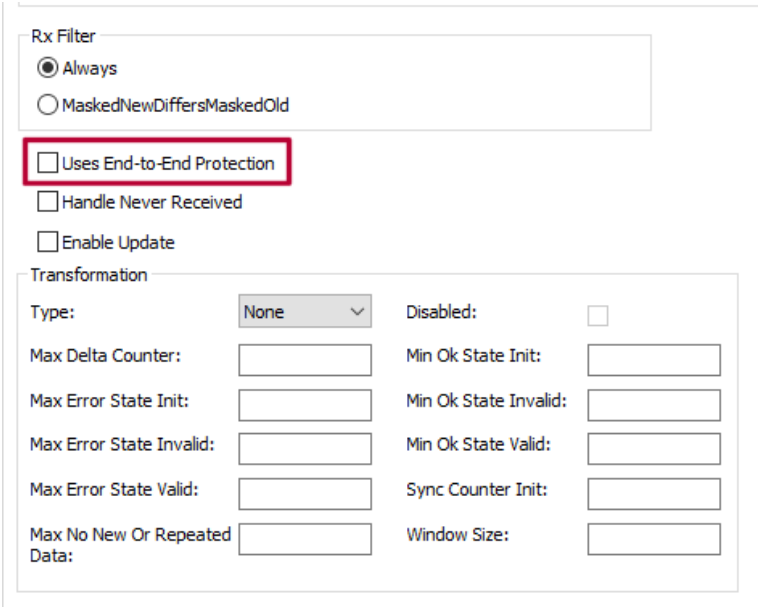


Figure 3-10 Port Prototype and Use End-to-End Protection

3.2 Code Generation

The code will only be generated by DaVinci Configurator Classic. Be sure that the Rte can be generated without any issues before trying to generate the transformers. E2EXf and ComXf can then be generated by DaVinci Configurator Classic (even if no containers inside the module are configured). The following files will then be added to your **GenData** folder:

> E2EXf_LCfg.c / E2EXf_LCfg.h / E2EXf_MemMap.h

> ComXf.c / ComXf.h / ComXf_MemMap.h / ComXf_Compiler_Cfg.h

3.2.1 Tx Side

<SWC_Template>.c

```
* Output Interfaces:
* ==================
*   Explicit S/R API:
*   -----------------
*   Std_ReturnType Rte_Write_[redacted](const REC_[redacted] *data)
*
```

Figure 3-11 Code Example of Generated Template Code

Rte_<SWC>.h

```
/**********************************************************************************************
 * Rte_Write_<p>_<d> (explicit S/R communication with isQueued = false)
 **********************************************************************************************
#  define Rte_Write_[redacted] Rte_Write_[redacted]
```

Figure 3-12 Code Example of Generated Template Code (Rte_write of Swc)

Rte_<OsApplication>.c

```
FUNC(Std_ReturnType, RTE_CODE) Rte_Write_[redacted](P2CONST(REC_[redacted]
{
  Std_ReturnType ret = RTE_E_OK;
  uint16 lengthBuffer;
  Std_ReturnType retTransformer;
  uint8 transformationBuffer_0[4];

  retTransformer = ComXf_SWC_[redacted](&(transformationBuffer_0[0]), &lengthBuffer, data);
  if (!Rte_IsHardTransformerError(retTransformer)) /* [redacted]_RTE_UNSUPPORTED_TRANSFORMER */
  {
    retTransformer |= E2EXf_SWC_[redacted](&(transformationBuffer_0[0]), &lengthBuffer, lengthBuffer);
  }
  if (!Rte_IsHardTransformerError(retTransformer)) /* [redacted]_RTE_UNSUPPORTED_TRANSFORMER */
  {
    ret |= Com_SendSignalGroupArray(ComConf_ComSignalGroup_[redacted], &transformationBuffer_0[0]);
    if ((ret == RTE_E_OK) && (retTransformer != RTE_E_OK)) /* COV_RTE_UNSUPPORTED_TRANSFORMER */
    {
      ret = RTE_E_SOFT_TRANSFORMER_ERROR;
    }
  }
  else
  {
    ret = RTE_E_HARD_TRANSFORMER_ERROR;
  }

  return ret;
} /* [redacted]
```

Figure 3-13 Code Example of Generated Template Code (Rte_write of OsApplication)

3.2.1.1 For the ComXf Function

ComXf.h

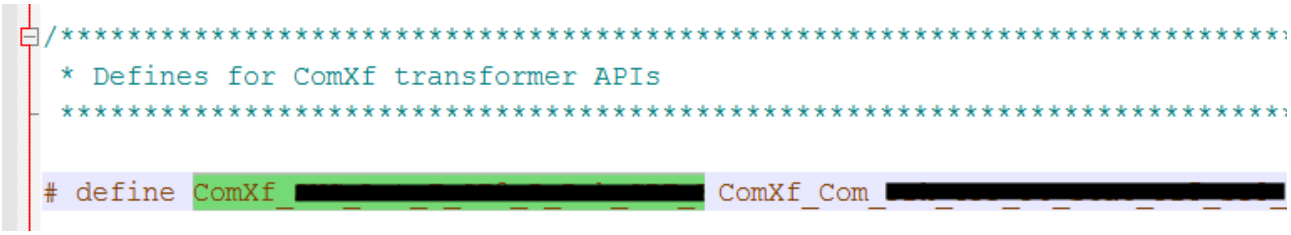


Figure 3-14 Code Example of Generated Template Code (ComXf Function)

ComXf.c



Figure 3-15 Code Example of Generated Template Code (ComXf Function Implementation)

3.2.1.2 For the E2EXf Function

E2EXf_LCfg.h

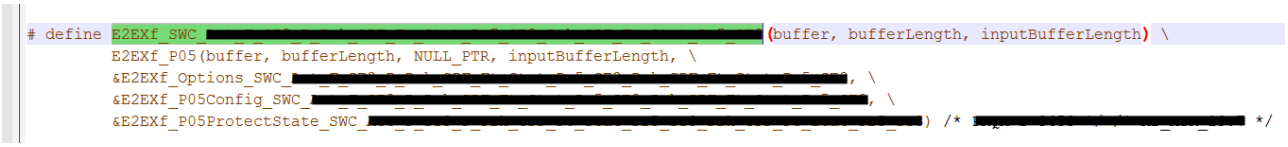


Figure 3-16 Defined Sub-Functions Inside of E2EXf

All the sub-functions are defined inside the E2E_LCfg.c (without screenshot).

3.2.2 Rx Side

<SWC_Template>.c

```
 * Input Interfaces:
 * =================
 *   Explicit S/R API:
 *   -----------------
 *   Std_ReturnType Rte_Read_[redacted](REC_[redacted] *data)
 *   Std_ReturnType Rte_Read_[redacted](REC_[redacted] *data)
 *   boolean Rte_IsUpdated_[redacted](void)
 *   boolean Rte_IsUpdated_[redacted](void)
 *
```

Figure 3-17 <SWC_Template>.c

Rte_<SWC>.h

```
/**********************************************************************************************************************
 * Rte_Read_<p>_<d> (explicit S/R communication with isQueued = false)
 *********************************************************************************************************************/
#  define Rte_Read_[redacted] Rte_Read_[redacted]
#  define Rte_Read_[redacted] Rte_Read_[redacted]
```

Figure 3-18 Rte_<SWC>.h

Rte_<OsApplication>.c

```
FUNC(Std_ReturnType, RTE_CODE) Rte_Read_[redacted](P2VAR(REC_[redacted], AUTOMATIC, RTE_C
{
  Std_ReturnType ret = RTE_E_OK;
  Std_ReturnType retTransformer;
  uint16 lengthBuffer;
  uint8 transformationBuffer_0[10];

  Rte_DisableOSInterrupts(); /* [redacted] */
  ret |= Com_ReceiveSignalGroupArray(ComConf_ComSignalGroup_[redacted], &transformationBuffer_0[0]);
  Rte_OsAppl_QM_C0_RxUpdateFlags.Rte_RxUpdate_CRC_[redacted] = Rte_OsAppl_QM_C0_RxUpdateFlags.Rte_RxUpdate_CRC_[redacted]
  Rte_EnableOSInterrupts(); /* [redacted] */
  lengthBuffer = sizeof(transformationBuffer_0);
  retTransformer = E2EXf_Inv_CRC_[redacted](&(transformationBuffer_0[0]), &lengthBuffer, lengthBuffer);
  if (!Rte_IsHardTransformerError(retTransformer)) /* [redacted]RTE_UNSUPPORTED_TRANSFORMER */
  {
    retTransformer |= ComXf_[redacted](&(transformationBuffer_0[0]), lengthBuffer, data);
  }
  if (Rte_IsHardTransformerError(retTransformer))
  {
    ret = RTE_E_HARD_TRANSFORMER_ERROR;
  }
  else if ((ret == RTE_E_OK) && (retTransformer != RTE_E_OK)) /* [redacted]_RTE_UNSUPPORTED_TRANSFORMER */
  {
    ret = RTE_E_SOFT_TRANSFORMER_ERROR;
  }

  return ret;
} /* [redacted] */
```

Figure 3-19 Rte_<OsApplication>.c

3.2.2.1 For the E2EXf Function

E2EXf_LCfg.h

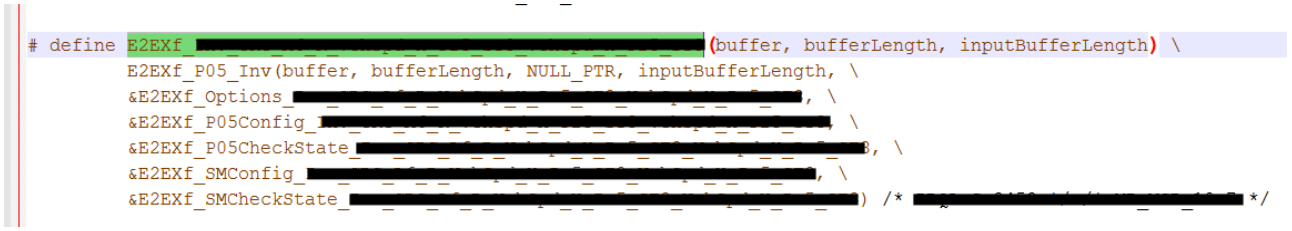


Figure 3-20 E2EXf_LCfg.h

All the sub-Functions are defined inside the E2E_Lcfg.c file (No screenshot).

3.2.2.2 For the ComXf Function

ComXf.h

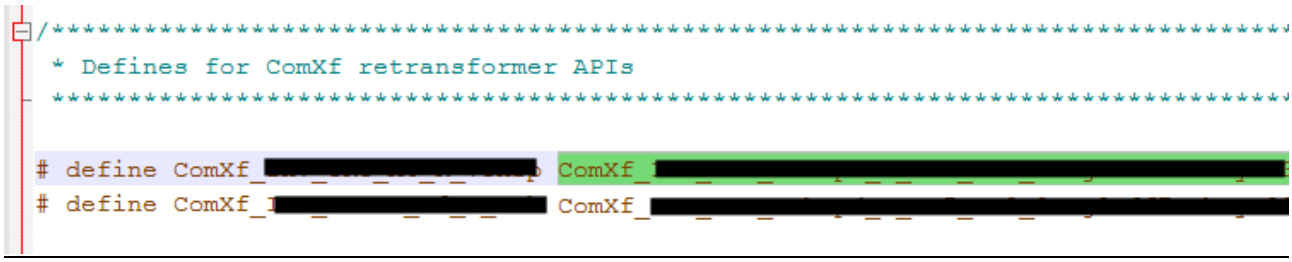


Figure 3-21 ComXf.h

ComXf.c

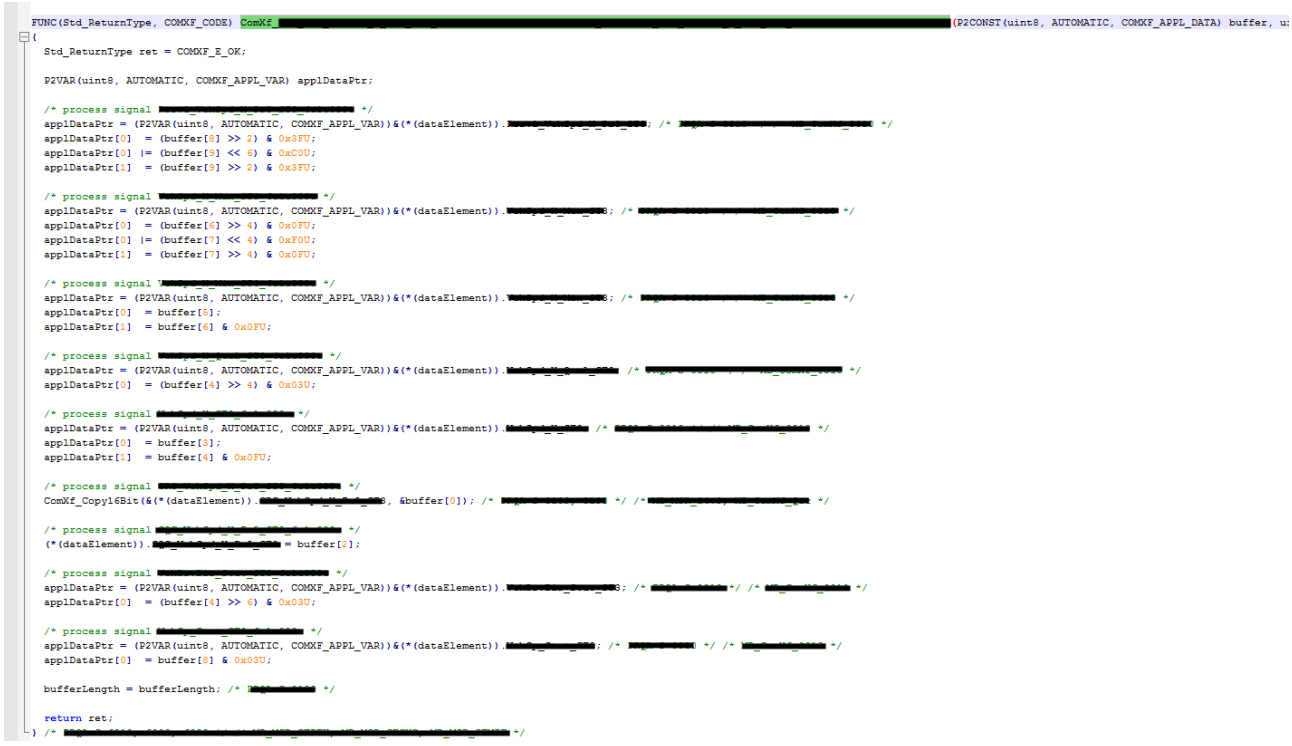


Figure 3-22 ComXf.c

3.3 Transformer Error Handling – Return Value

Caution
This chapter is only relevant for Release R18 or newer.

The E2E Return Value will be packed as parameter inside the `Rte_Read()` function call:

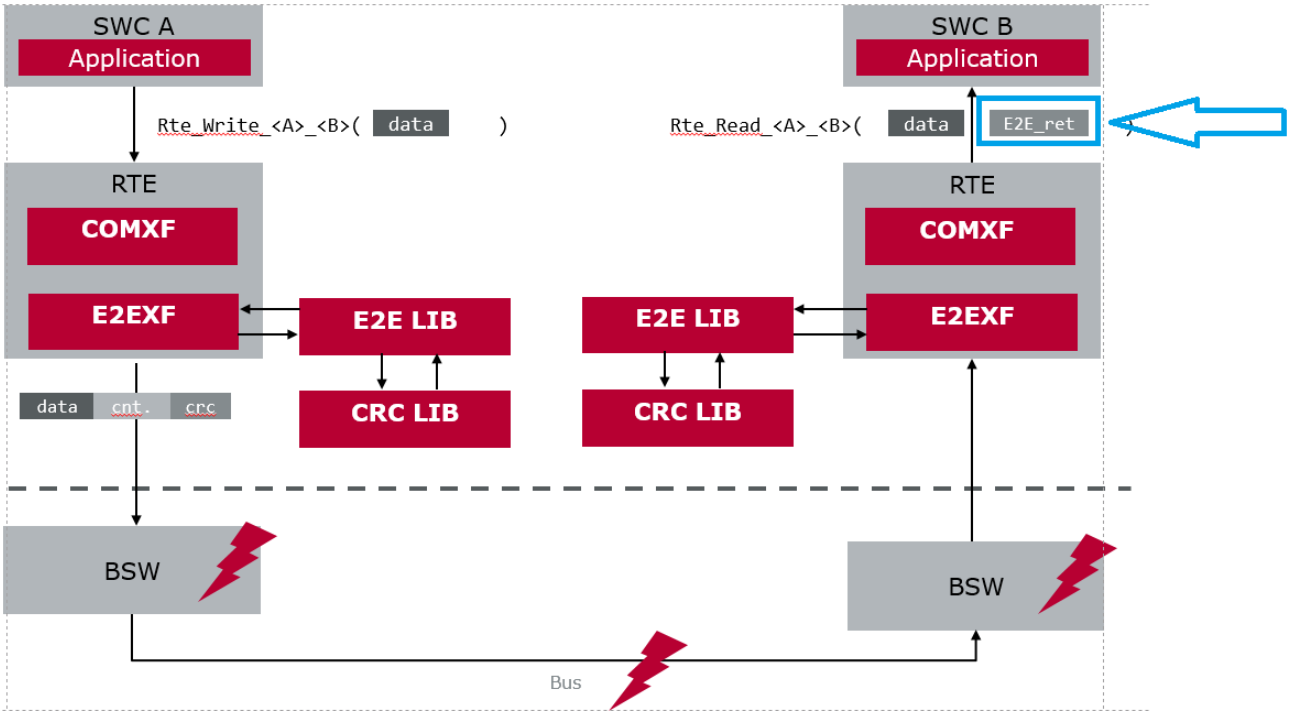


Figure 3-23 Schematic Overview

Reference

You have the possibility on Rx side to add the return value of the E2E transformation as parameter as specified in the **TechnicalReference_Rte.pdf**.

Prototype

```
Std_ReturnType Rte_Read_<p>_<d> ( [IN Rte_Instance instance,] OUT <DataType> *data
[, OUT Rte_TransformerError transformerError] )
```

Parameter

instance	Instance handle, used to distinguish between the different instances in case of multiple instantiation. Note: This is an optional parameter depending on the configuration of `supportsMultipleInstantiation` attribute.
*data	The output <data> is passed by reference. The <DataType> is the type, specified at the data element prototype in the SWC description.
transformerError	Optional OUT parameter if `transformerErrorHandling` is enabled.

Return code

RTE_E_OK	Data read successfully.
RTE_E_UNCONNECTED	Indicates that the receiver port is not connected

Figure 3-24 API Table Technical Reference RTE

To activate it, **Transformation Error Handling** must be activated on **Port API Options** of the dedicated **Application Port**.

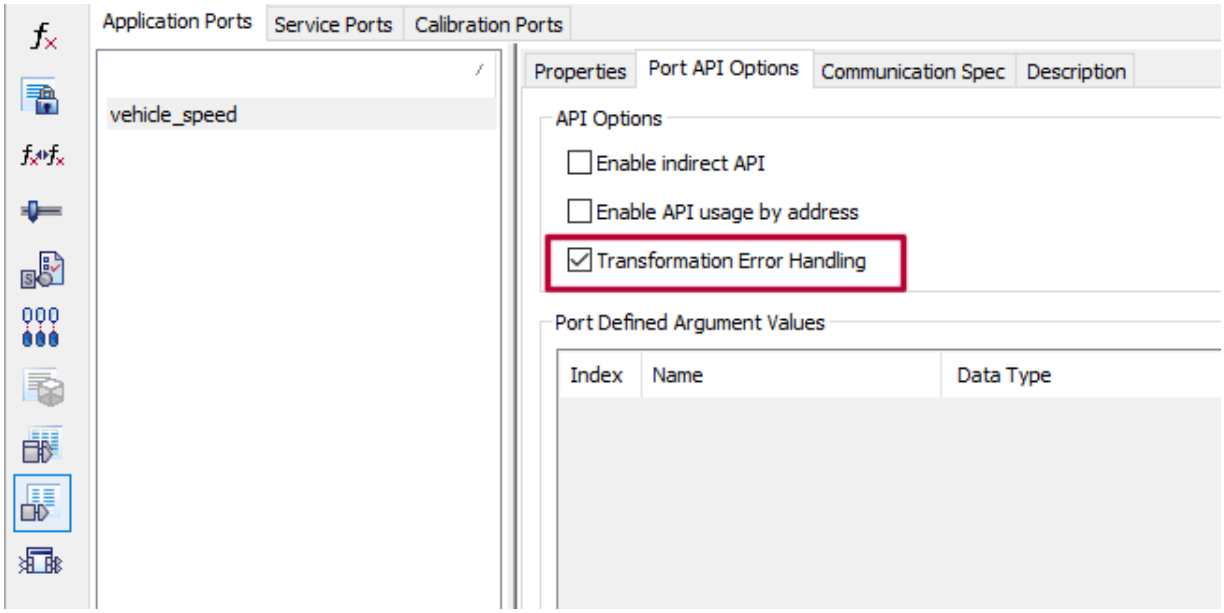


Figure 3-25 Transformation Error Handling in DaVinci Developer Classix

The generated code now includes the return value:

```
* Input Interfaces:
* ================
*   Explicit S/R API:
*   -----------------
*   Std_ReturnType Rte_Read_[redacted](REC_[redacted] *data)
*   Std_ReturnType Rte_Read_[redacted](REC_[redacted] *data, Rte_TransformerError *transformerError)
*   boolean Rte_IsUpdated_[redacted](void)
*   boolean Rte_IsUpdated_[redacted](void)
```

Figure 3-26 Generated Code

3.4 Abbreviations

Abbreviation	Description
AR	Autosar
ARXML	Autosar xml, typical file ending for AUTOSAR description files using a xml scheme defined by AUTOSAR consortium (and described in the specifications)
E2E	End-To-End / Ende-zu-Ende
E2ePw	E2E Protection Wrapper
CP	Classic Platform
LDF	LIN Description File
VSDE	Vector System Description extension
VLC	Vector Legacy Converter
XML	Extensible Markup Language, very common rule based encoding of documents. It is meant to create file formats, which are processable/readable by human and computer.

4 Glossary and Abbreviations

4.1 Glossary

Term	Description
Configuration data	Data that is used to adapt the MICROSAR Classic Safe component to the specific use-case of the user of MICROSAR Classic Safe.
Generated code	Source code that is generated as a result of the configuration in DaVinci Configurator Classic.
Safety Manual	The Safety Manual details the requirements for the user of MICROSAR Classic Safe for integration of MICROSAR Classic Safe into the item development.

4.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basic Software
ECU	Electronic Control Unit
MICROSAR Classic	Microcontroller Open System Architecture (the Vector AUTOSAR solution) Classic
PPORT	Provide Port
RTE	Runtime Environment
RPORT	Require Port
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification

5 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com